

Essence Explorer – a Tool for Researchers

An Open-Source Prolog and AI Implementation

Kim Horn

Version 1.02

20 March 2024





- Goals
- Background – what is Essence ?
- Explorer Features
- Web GUI Views:
 - Cards
 - Relationship Viewer – Graphical Framework
- Prolog and the Essence API
- AI Research – Chat with an Essence Assistant

‘IJ’ is used in this pack to refer to Ivar Jacobsen International.

‘IJ’ provide two commercial products: Teamspace and WorkBench.

‘IJ’ provides paid consulting on Essence that uses these products.

Copyright Notice, material in this pack is from:

- Essence material is property of Ivar Jacobsen International and Semat.org
- The Essence Standard is provided by OMG
- www.semat.org



www.ivarjacobson.com



Goals

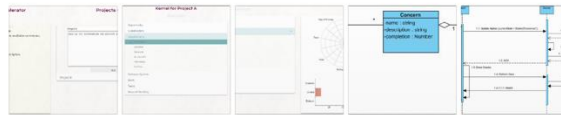


- 1) One of the goals of Semat is to provide Essence as a tool to test theories in Software Engineering Research. The author wished to research things that the IJI tools do not support:
 - **'Roles' as composites** from 'Competencies', as first-class elements not Patterns.
 - **Visually explore** element relationships and associations.
 - **Translation: Produce** generate English descriptions of Essence and Practices, for:
 - Describing, checking and summarising practices for humans.
 - Generate Java, SQL, Python and Cypher versions of the Essence models.
 - **AI research:** As input to Generative AI LLM Chatbots. Allow people to ask Chatbot about Essence and a Practice.
- 2) There is much published research on Essence, testing theories, and it seems each paper develops its own tooling, as they claim there is a lack of tooling. The current author has also found this to be true.
- 3) It would be beneficial to help support and foster a *free open-source* tool for:
 - Standardise Research, educational and learning use;
 - Help small companies start with Essence with low entry cost initially, e.g., free software tool.
- 4) This tool would *benefit the research community* and also help get Essence used more. Features:
 - Known quality, and a single primary representation of Essence (**A Prolog Knowledge Base**);
 - Up to date and aligned with Essence OMG Standard;
 - Based on a minimal set of free Open-source frameworks, just Javascript and Prolog;
 - Prolog provides a logic language to test and check 'research' theories, and algorithms;
 - Allow anything to be coded and tested;
 - Open source managed, researchers contribute, and guide in the direction they need.

Many Research Papers, e.g. Search for 'Essence' in ResearchGate



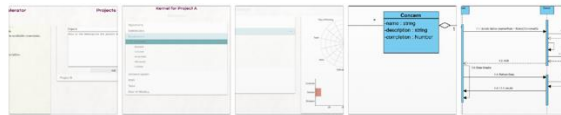
ResearchGate



A Web-based modeling tool for the SEMAT Essence theory of software engineering

Article Sep 2013 · DOI: 10.5334/jors.ad · ISBN: 2049-9647

Daniel Graziotin · Pekka Abrahamsson



A Web Positioning System for SEMAT Essence theory of Software Engineering

Article Jul 2013

Daniel Graziotin · Pekka Abrahamsson

A brief TOGAF description using SEMAT Essence Kernel

Preprint Sep 2019

David C. Múnera · Fernán A. Villa G



Source



State-Based Monitoring and Goal-Driven Project Steering: Field Study of the SEMAT Essence Framework

A Case Study of Software Engineering Methods Education Supported By Digital Game-Based Learning—Applying the SEMAT Essence Kernel in Games and Course Projects

Conference Paper Apr 2017 · DOI: 10.1109/EDUCON.2017.7943076

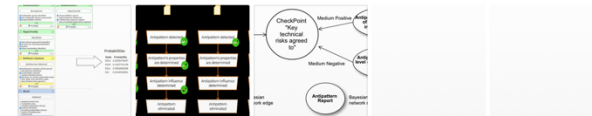
Jöran Pieper · Oliver A. Lüth · Michael Goedicke · Peter Forbrig



Representación en el Núcleo de la Esencia de Semat de las Competencias de un Equipo de Desarrollo de Software

Article Aug 2019 · DOI: 10.4067/S0718-07642019000400217 · ISBN: 0718-0764

Claudia E. Durango-Vanegas · Carla M. Zapata-Rueda · Carlos Mario Zapata-Jaramillo



Implementation of Essence Practice into Bayesian Networks

Article Sep 2022 · DOI: 10.55549/epstem.1176067 · ISBN: 2602-3199

Oleg A. Zmeev · D. O. Zmeev · L. S. Ivanova

Implementation of Essence Practice into Azure DevOps Server System

Article Dec 2020 · DOI: 10.17587/prin.11.311-321

Oleg A. Zmeev · D. O. Zmeev · A. N. Danilenko



- Many researchers build their own one off tools to implement their research.
- Number of add hoc implementations.
- Unclear of the quality and how up to date these are ?
- Research, on the right, suggests:

“The goal of Essence is to develop a theoretically sound basis for software engineering practice and its wide adoption. However, Essence is far from reaching academic- and industry-wide adoption. The reasons for this include a struggle to foresee its utilization potential and a lack of tools for implementation. “

SOFTWARE PAPER

A Web-based modeling tool for the SEMAT Essence theory of software engineering

Daniel Graziotin¹ and Pekka Abrahamsson¹

¹ Free University of Bozen-Bolzano, Bolzano, Italy

As opposed to more mature subjects, software engineering lacks general theories that establish its foundations as a discipline. The Essence Theory of software engineering (Essence) has been proposed by the Software Engineering Methods and Theory (SEMAT) initiative. The goal of Essence is to develop a theoretically sound basis for software engineering practice and its wide adoption. However, Essence is far from reaching academic- and industry-wide adoption. The reasons for this include a struggle to foresee its utilization potential and a lack of tools for implementation. SEMAT Accelerator (SematAcc) is a Web-positioning tool for a software engineering endeavor, which implements the SEMAT's Essence kernel. SematAcc permits the use of Essence, thus helping to understand it. The tool enables the teaching, adoption, and research of Essence in controlled experiments and case studies.

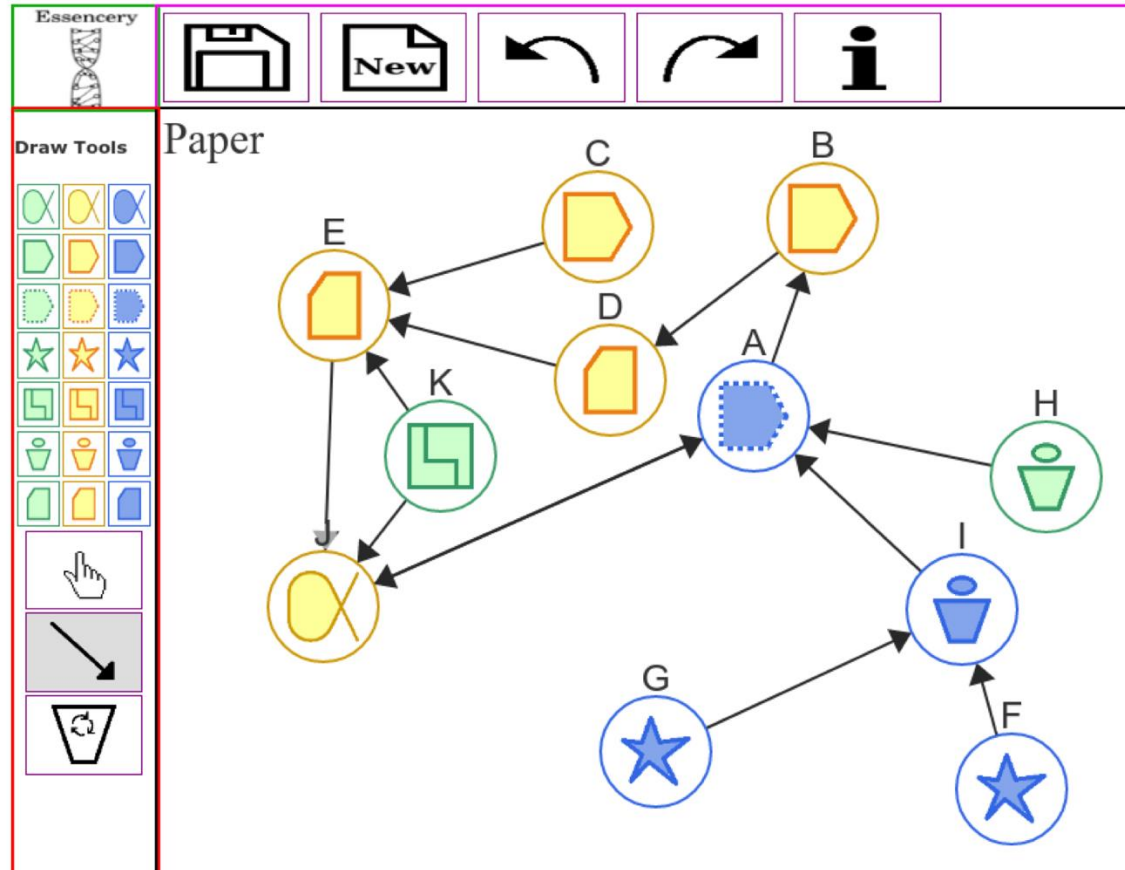
Example 2: Essencery – A Tool for Essentializing Practices



The authors state: “Essence supports the use of any development practices and provides a framework for building a suitable method for any software engineering context.

However, Essence presently suffers from low practitioner adoption that is partially considered to be caused by a lack of proper tooling. “

Essencery (2018) is a tool for essentializing software engineering methods and practices using the Essence graphical syntax.



IJI Products focus on limited markets, with fixed functionality.



The IJI products focus on particular niche markets, and do this well, but they are not end to end 'method' modelling tools, and do not allow research and tailoring of the views, logic and information. They provide fixed specific views for particular approaches;

- **Teamspace** focuses on 'Game playing' and does not cover all of the Essence Specification. Many Essence Elements, e.g. Associations, Element Relationships, cannot be displayed graphically in TeamRooms (workspaces):
 - It is difficult to create and edit Essence Elements.
 - The tool does not support iterative 'method' design or practice refactoring. Ideally your method is designed upfront, say in Workspace, then deployed in Teamspace .
 - It is not meant to be a method 'modelling' tool, but a workspace for interactive games, and facilitation.
 - The element editor displays information, but it is often difficult to work out where this information comes from. There is lack of traceability.
- **Workspace** provides Element editing but with limited visual modelling and is essentially a tool for creating Printed Card Packs and static HTML pages. Explorer provides dynamic pages, edited code is immediately viewable
 - It provided no graph like tools for editing and exploring elements their associations and relationships.
- **There still exists a gap in the marketplace;** a need for an 'Official Supported' Open-Source Research Version of Essence, that allows full end to end modelling, and experimentation:
 - A tool where anything can be changed and everything is traceable, from basic facts, to front end views;
 - Allow research on any part of the tool, with any data changed;
 - One that is aligned with standard and managed, with known quality;
 - One that allows all elements to be easily refactored, moved between practices, and evolve in a continuous improvement manner. Allows new knowledge and resources to be added with ease.



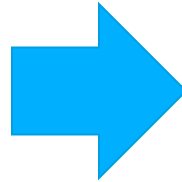
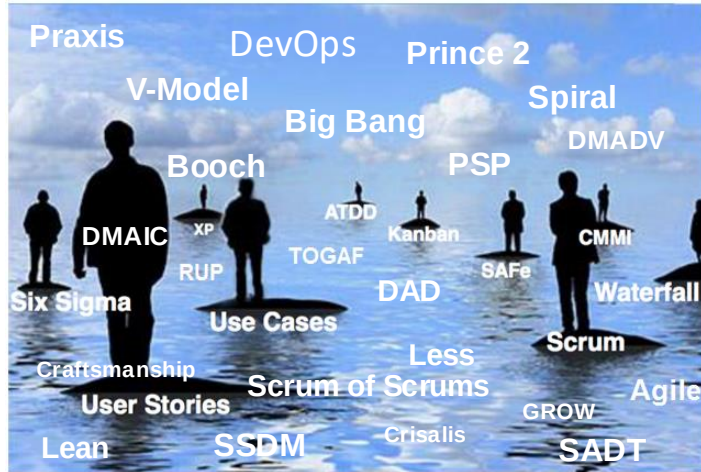
Background

A very brief intro to Essence

Where are we with software development 'Processes' ?



Everyone of us knows how to develop our own software, but as a community we have no widely accepted common ground; there are many, if not hundreds of thousands of 'processes'.



What I want to talk about is a body of work trying to achieve 3 goals:

- Goal 1: Is it possible to 'distil' an Essence of commonality, a Kernel ?
- Goal 2: Is there a better way to do things ?
- Goal 3: Can Software Engineering be based on a Theory ?



Software Engineering Method and Theory



Founded in September 2009 by:
Ivar Jacobson, Bertrand Meyer, Richard Soley

The Grand Vision

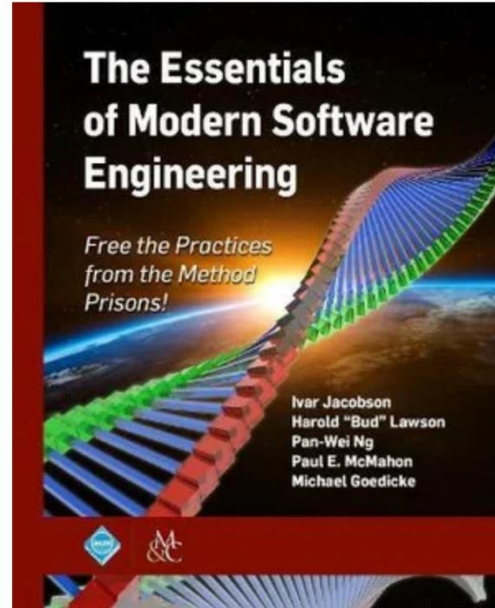
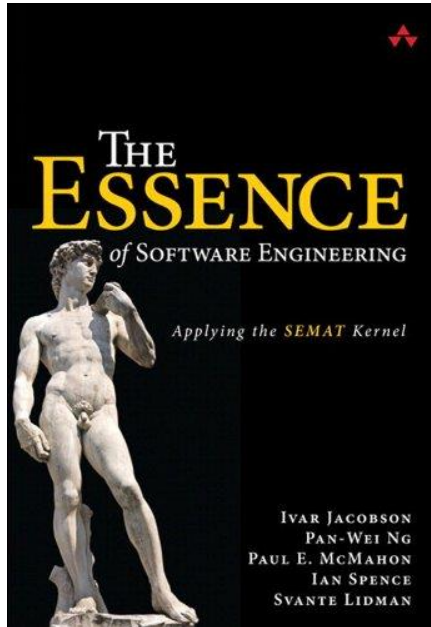
To support a process to refund software engineering,
based on a solid theory, proven principles and best
practices.

The OMG Standard, created in 2014

Essence – Kernel and Language for Software Engineering
Methods, V1.2



<https://www.omg.org/spec/Essence/1.2/PDF>

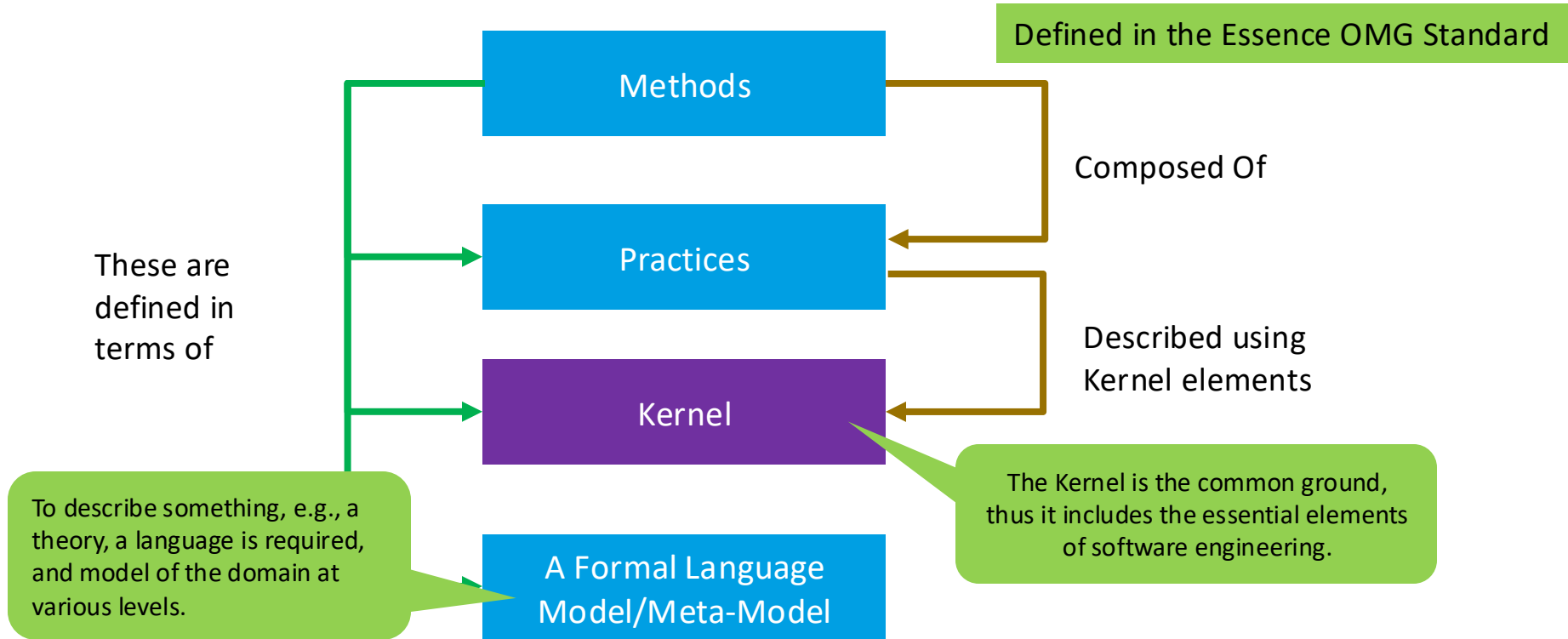


- Know as 'IJI'
- Consulting
- Change
- Training
- Licensed Commercial Software Products:
 - TeamSpace
 - Practice Workbench
- Practice Library

The Books

“The Essence of Software Engineering.”

“The Essentials of Modern Software Engineering.”



You can tailor make your own 'Methods' combining practices you want, but all based on the one Kernel

Four key things are all that's required; The Essentials



1. We are concerned about the ***Customer***, the ***Solution*** and the ***Endeavour***.
2. There are ***Things To Work With***, that we want to understand, monitor (progress and health), direct and control, called ***Alphas***.
3. There are things to do; ***Activities***.
4. We need ***Competency*** to do stuff well.

Essentially that's it. The minimal things we need to get right; a Kernel

Minimally 3 Key Concerns



Is everything to do with the actual use and exploitation of the software system to be produced

Customer

Is everything to do with the specification and development of the software system.

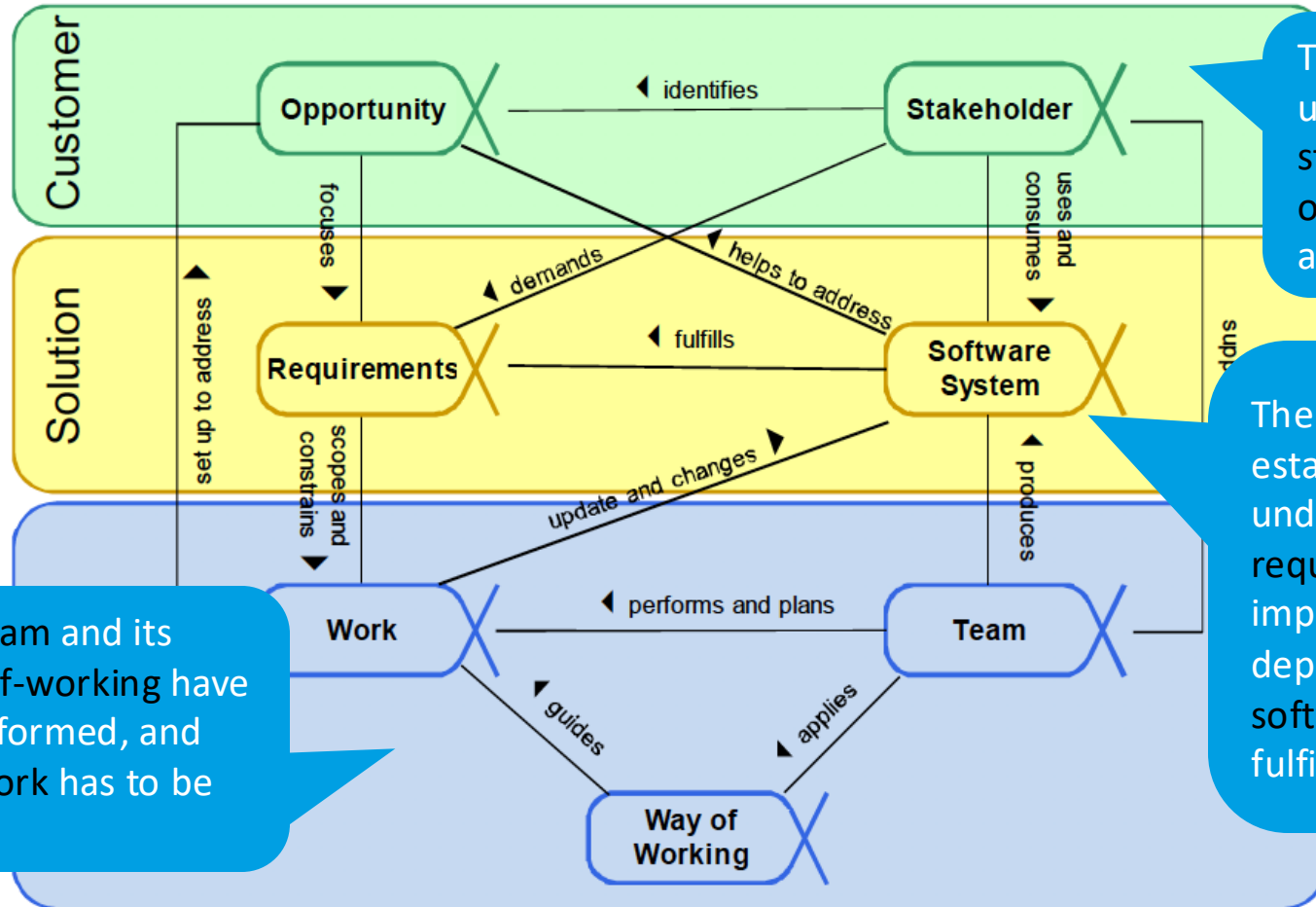
Solution

Is everything to do with the team, and the way that they approach their work.

Endeavour

Undertaking this ***Endeavour*** to create a ***Solution*** with and for the ***Customer***

There are 7 Essential “things to work with” – termed Alpha’s α

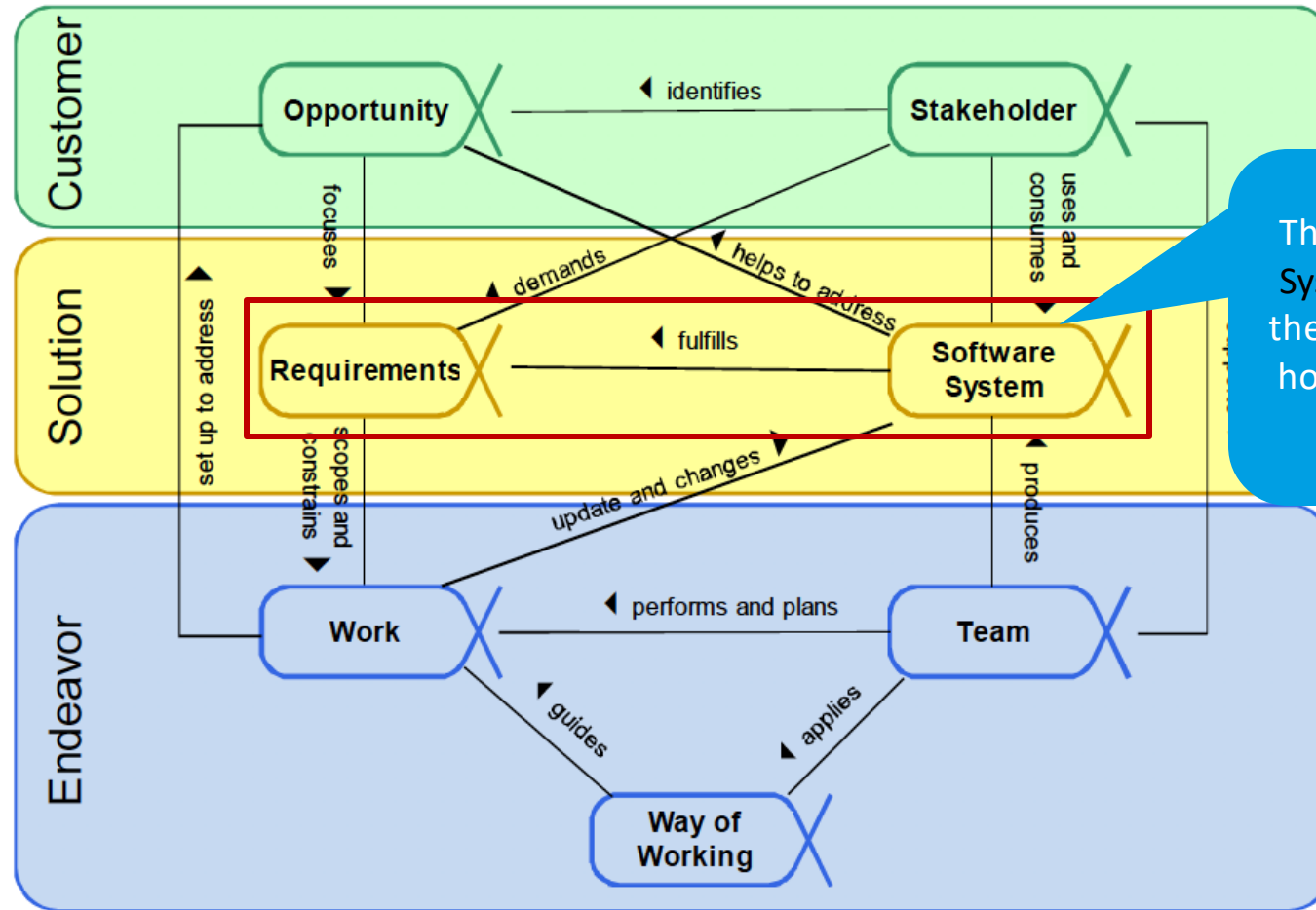


The team needs to understand the stakeholders and the opportunity to be addressed

The team needs to establish a shared understanding of the requirements, and implement, build, test, deploy and support a software system that fulfils them.

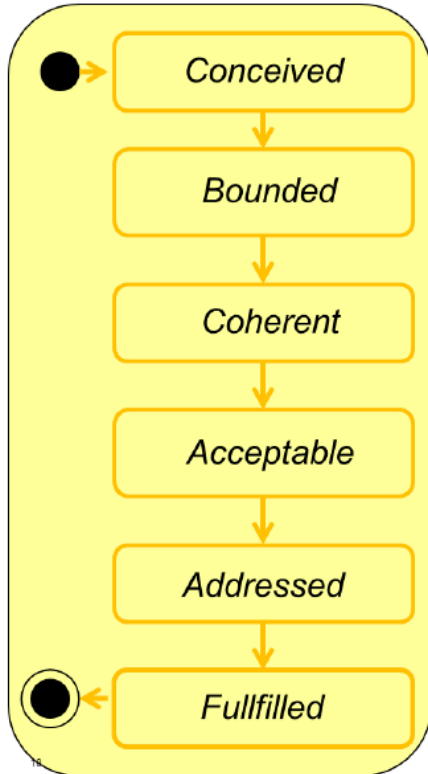
the team and its way-of-working have to be formed, and the work has to be done.

These Alpha's have important relationships



The goal is to build a System that 'Fulfills' the Requirements but how do we get there ?

To 'Fulfil' the Requirements Alpha's it goes through intermediate States



The need for a new system has been agreed.

The purpose and theme of the new system are clear.

The requirements provide a coherent description of the essential characteristics of the new system.

The requirements describe a system that is acceptable to the stakeholders.

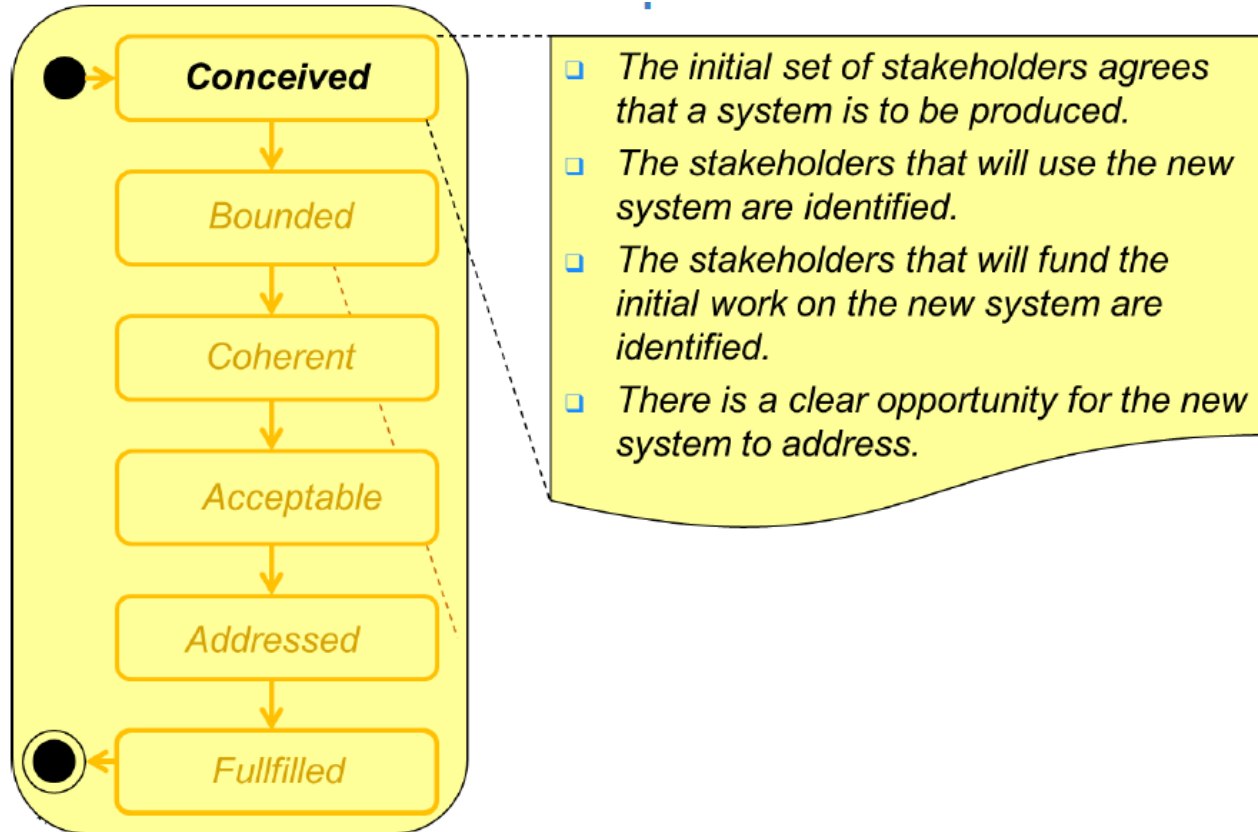
Enough of the requirements have been addressed to satisfy the need for a new system in a way that is acceptable to the stakeholders.

The requirements have been addressed to fully satisfy the need for a new system.

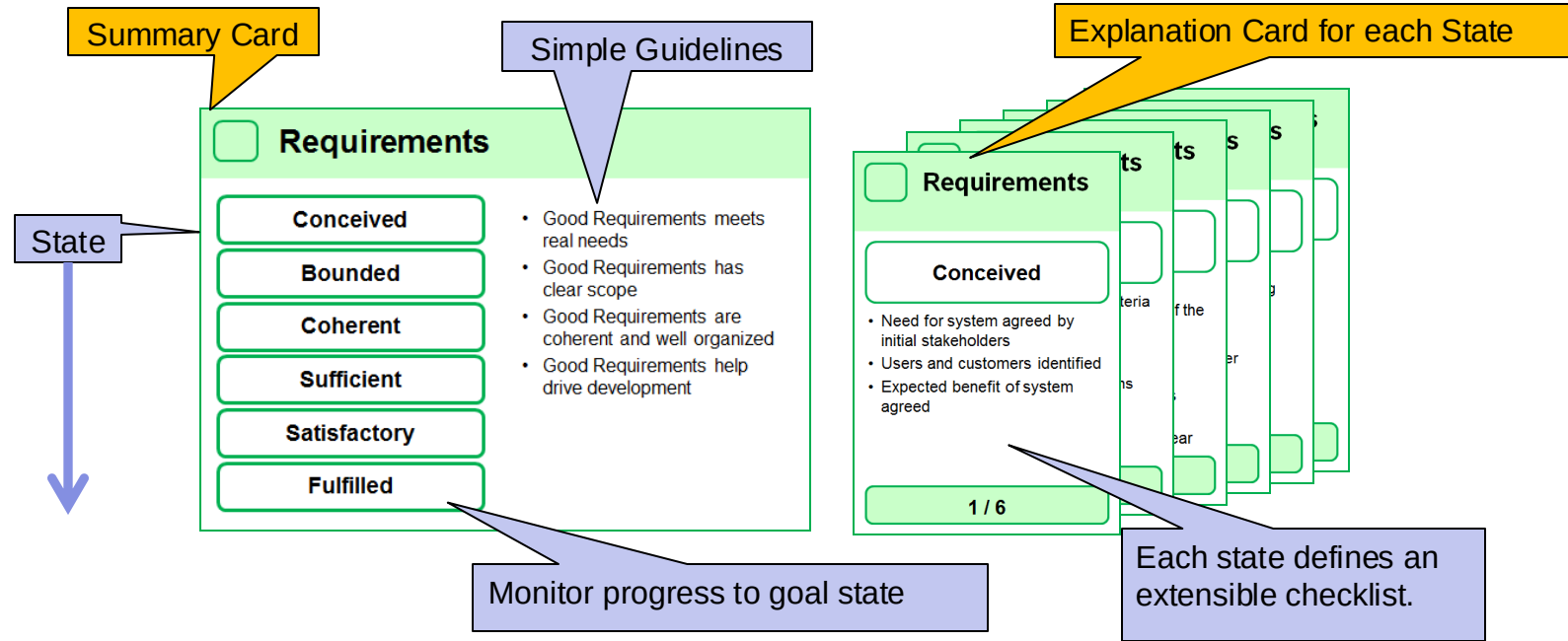
This is not suggesting a waterfall approach, or process, to requirements, indeed it could describe an iteration. Naturally, requirements go through these states.

- It does not specify how
- It does not specify what requirements are.
- These states are essential

Each State has a Checklist, completed to transition the state

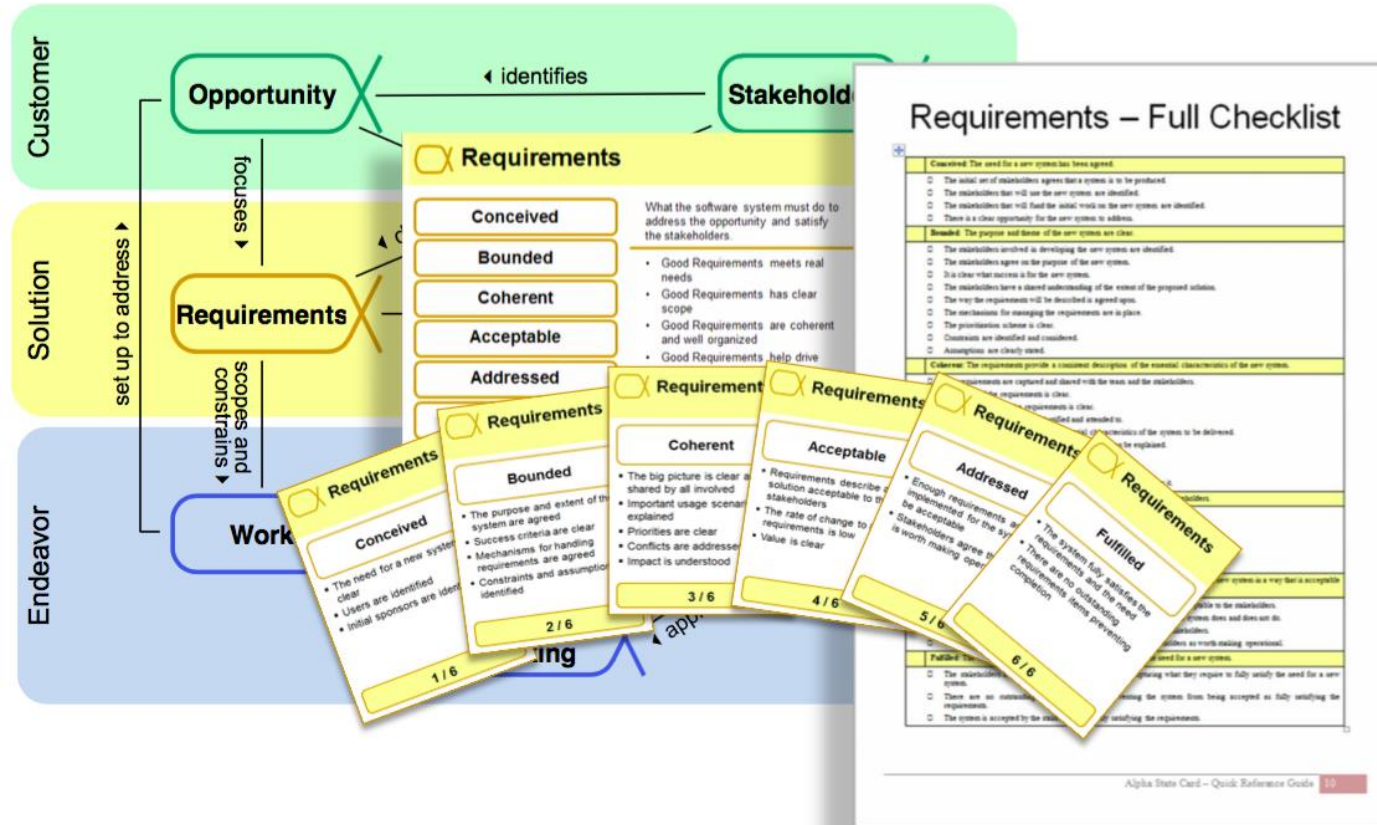


Descriptive tool: Essence Alpha Summary Cards and Detail Cards



Work moves items through states. For example, the goal of building a system is to get “*Conceived*” **Requirements** to a “*Fulfilled*” state, by executing **Work**, by the **Team**, to deliver an “*Operational*” **Software System**. We are interested in monitoring these items to plan work.

Cards and Checklists are Tangible, Physical





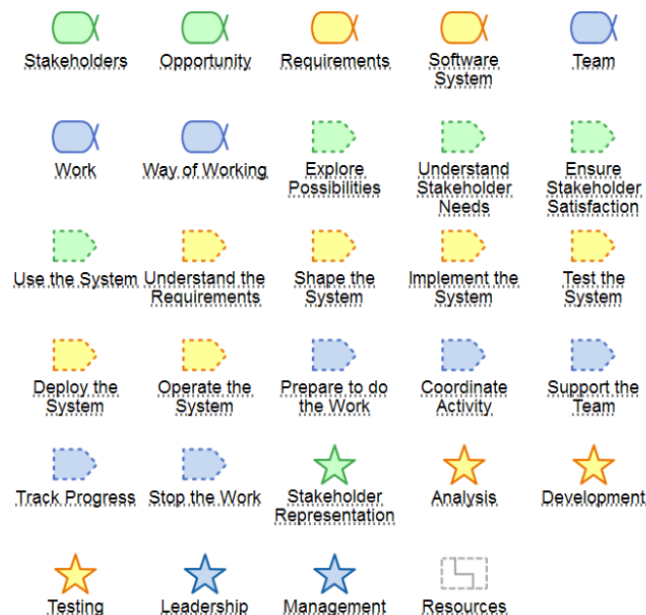
A view of the Kernel as a Summary Card, includes:

- Alphas
- Activity Spaces
- Competencies
- Patterns

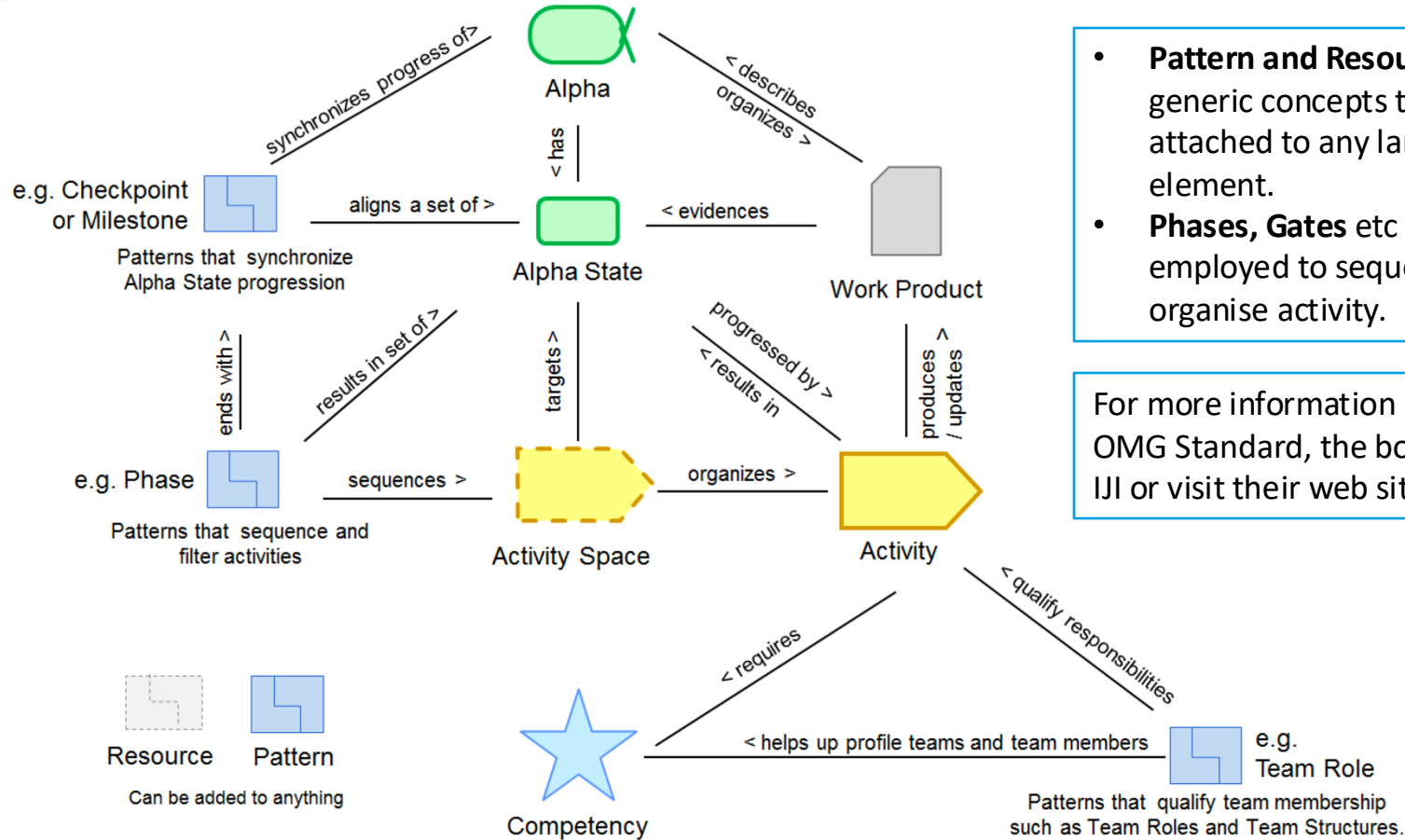
New Practices can use these elements and include new ones.

Essence Kernel

The essence of effective, scalable software engineering.



There is much more than just the Kernel, for example



- **Pattern and Resource** are generic concepts that can be attached to any language element.
- **Phases, Gates** etc can be employed to sequence and organise activity.

For more information refer to the OMG Standard, the books, or ask IJI or visit their web site.



Explorer Features



Work on this started in about 2015, and slowly evolved. Originally just a Prolog DB of the Essence Spec. Recently the API and GUI was added.

- Full Model of Essence Kernel in Prolog. Full model of other Spec elements: Activities, Work Product, Associations etc.
 - The model is close to the OMG Standard data model, but not identical.
 - An OO model is used, with Inheritance and Instantiation (Alphas and Work Products).
- Create Practices, with their new elements, as modules, provide these by default: Scrum, Architecture, User Story, Use Cases.
- Add in new elements, e.g., 'Sprint Alpha, 'Sprint Backlog Work Product', etc...
 - Research and create new Element types, e.g., a new Type 'Role'.
 - Validate these elements; is the data OK, consistent, what items are missing, etc.

Web User Interface (GUI):

- Web Browser to Explore and view all Elements, their Associations and all details interactively. These are dynamic pages not static.
- Show Elements Filtered by Practices / Kernel, e.g., Scrum, Use Case etc.
- Select Element Types and see them listed by Concern, e.g.,
 - Alphas show States to the Right, Work Products show Details to Right.
 - Activities show impacted Elements to right (Alphas and Work products).
 - Alpha State and Work product Detail CheckBox check states are stored by a Project.
- Load and Save Projects that record the teams latest CheckBox States, and Alpha Instances.
- Provide framework to Graphically draw elements and their associations and relationships.

REST JSON API

- HTTP API exposes the Prolog Model to the Web User Interface, or other systems.
- Allows any system to use the server and access Essence.
- There is no separate Data Base Server as Prolog models all the data, state, and all the logic.

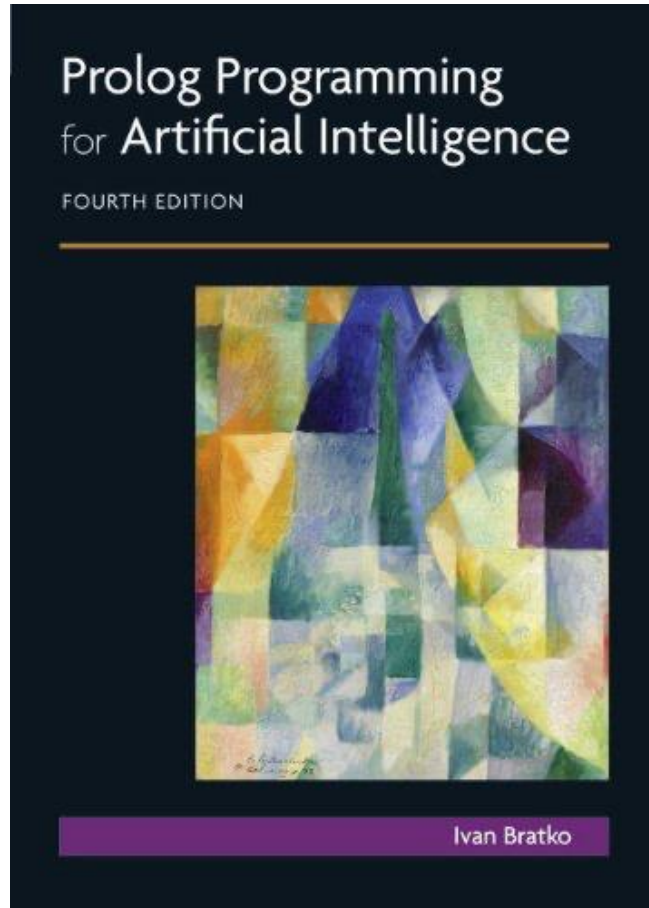
Why Prolog ?



- Prolog is a logic programming language and has its roots in first-order logic. Historically an AI and Natural Language Processing language. This makes AI research very easy.
- Unlike many other programming languages, it is primarily a declarative language: the program logic is expressed in terms of relations, represented as facts and rules. The declarative nature is ideal for AI research.
- It supports Domain Specific Languages and Constraints.
- It provides an elegant way to store data, query it, report on it, check it, and apply logic.
 - No separate data base is required as Prolog is a data base.
 - It provide an HTTP server with JSON REST APIs, in a few lines of code.
 - Program = Data; Prolog does not distinguish Program from Data.
 - Datalog is a standard subset of Prolog for just Data.
 - Easy to add new elements, types, reports, logic, new views etc
- Other large data base endeavours are supplied in Prolog form, e.g., **prologdb(5WN)** is WordNet in Prolog: <https://wordnet.princeton.edu/documentation/prologdb5wn>
- Prolog is an ISO Standard and has a de-facto **Edinburgh** Prolog standard.
- SWI-Prolog is a comprehensive, robust and freely available across most operating systems, and runs in docker.
- As such, Prolog is ideal language to express and manipulate Essence.



SWI Prolog



Prolog as a core to Essence allows AI to be applied and researched easily. This is a great book for Prolog and AI.

The fourth edition of this best-selling guide to Prolog and Artificial Intelligence has been updated to include key developments in the field while retaining its lucid approach to these topics.

Author:

Professor Ivan Bratko leads the AI Lab in the Faculty of Computer and Information Science at the University of Ljubljana.

He has taught Prolog worldwide, as well as applied Prolog in medical expert systems, robot programming, qualitative modelling and computer chess research.



Web GUI

Essence Explorer Web GUI



Search, Filter, Validate
Load/Save here

Select Element Types
Here

Concern Window
Select Elements in
Concerns

See the Elements in
the Accordions Here.
You can also select an
Element here from
dropdown.

Essence Explorer

FileFilterValidateHelp

Search

Search

Choose an Element:

☒ Alpha

☐ Competencies/Roles

☐ Activity Spaces

☐ Activities

☐ Work Product

☐ Pattern

Collapse All

Customer

⌵ Opportunity

⌵ Stakeholders

Solution

⌵ Requirements

⌵ Software System

⌵ User Story

⌵ Product Goal

⌵ Product Backlog Item

Endeavour

⌵ Work

⌵ Way of Working

⌵ Team

⌵ Improvement

⌵ Sprint

⌵ Sprint Goal

Alpha

⌵

Choose Alpha

Show All

Competency / Roles

⌵

Activity Space

⌵

Activity

⌵

Work Product

⌵

Patterns

⌵

Explore an 'Endeavour' Concern Alpha 'card', and its States



Use chose an Alpha

Selected 'Team' in
Endeavour Concern



See the Team card,
with States, Points
and Associations.

Select State 'Seeded'
and explore the
CheckBoxes on the
Right

The screenshot shows the 'Essence Explorer' application interface. At the top, there is a blue header with the title 'Essence Explorer' and navigation links: 'File', 'Filter', 'Validate', and 'Help'. A search bar is located on the right. Below the header, a section titled 'Choose an Element:' contains radio buttons for 'Alpha', 'Competencies/Roles', 'Activity Spaces', 'Activities', 'Work Product', and 'Pattern'. A 'Collapse All' button is also present. The main content area is divided into three horizontal sections: 'Customer' (with 'Opportunity' and 'Stakeholders'), 'Solution' (with 'Requirements', 'Software System', 'User Story', 'Product Goal', and 'Product Backlog Item'), and 'Endeavour' (with 'Work', 'Way of Working', 'Team', 'Improvement', 'Sprint', and 'Sprint Goal'). The 'Team' card in the 'Endeavour' section is highlighted with a red border. Below this, the 'Alpha' section is shown, featuring a 'Choose Alpha' dropdown and a 'Show All' button. The 'Team' card is displayed with a list of states: 'Seeded', 'Formed', 'Collaborating', 'Performing', and 'Adjournd'. The 'Seeded' state is selected, and its details are shown on the right, including a description and a list of checkboxes: 'Team's mission is clear', 'Team knows how to grow to achieve the mission', 'Required competencies are identified', and 'Team size is determined'. At the bottom, the 'Associations' section lists: 'Way of Working - Team applies the way of working', 'Work - Team performs and plans work', and 'Software System - Team produces software_system'.

Save the State of a Projects Alpha



Load a Project

Choose 'Opportunity'

Selected the first two
CheckBoxes as ticked,
on the Right

The screenshot displays the 'Essence Explorer' application interface. At the top, there is a menu bar with 'File', 'Filter', 'Validate', and 'Help'. A search bar on the right contains 'test case' and a 'Search' button. Below the menu, a 'Choose an Element' dropdown menu is open, showing options: 'Load Project', 'Load Practice', 'Alpha' (selected), 'Compete', 'Remove Practice', 'Activity Spaces', 'Activities', 'Work Product', 'Pattern', 'Collapse All', and 'Save'. The main workspace is divided into three horizontal sections: 'Customer' with 'Opportunity' and 'Stakeholders' buttons; 'Solution' with 'Requirements', 'Software System', 'User Story', 'Product Goal', and 'Product Backlog Item' buttons; and 'Endeavour' with 'Work', 'Way of Working', 'Team', 'Improvement', 'Sprint', and 'Sprint Goal' buttons. Below these is a section titled 'Alpha' with a 'Choose Alpha' dropdown and a 'Show All' button. The 'Opportunity' state is selected, showing a definition: 'The set of circumstances that makes it appropriate to develop or change a software system'. A list of characteristics follows: 'A good opportunity is identified addressing the need for a software-based solution', 'A good opportunity has established value', 'A good opportunity has a software-based solution that can be produced quickly and cheaply', and 'A good opportunity creates a tangible benefit'. An 'Associations' section lists 'Requirements - Opportunity focuses Requirements'. A red arrow points to the right, where a detailed view of the 'Opportunity' state is shown. This view includes a header 'Opportunity' and a section 'Identified' with three checkboxes: 'Opportunity identified that could be addressed by a software-based solution' (checked), 'A stakeholder wishes to make an investment in better understanding potential value' (checked), and 'Other stakeholders who share opportunity identified' (unchecked).

31



α	Stakeholders
Recognized	The people, groups, or organizations who effect or are affected by a software_system - Healthy stakeholders represent groups or organizations affected by the software_system - Healthy stakeholder representatives carry out their agreed to responsibilities - Healthy stakeholder representatives cooperate to reach agreement - Healthy stakeholders are satisfied with the use of the software_system
Represented	
Involved	
In Agreement	
Satisfied for Use	

Selected

Formed

Collaborating

Performing

Adapted

Team

The group of people actively engaged in the development, maintenance, delivery and support of a specific software system.

- A healthy team meets to team goals effectively
- A healthy team has members that collaborates effectively
- A healthy team focus on their work
- A healthy team continually improves

Competency and Role Cards: Role is aggregate of Competencies



Choose the 'Competency / Role' Tab.
The Competencies and Roles are shown in their concern.

Select 'Stakeholder Representation'

Note 'Scrum' is added as a Competency. How can we have a 'Scrum Master' role with no scrum competency.

Choose Role 'Scrum Master'
that composes competencies:

- Management – L3
- Scrum – L3
- Leadership – L3

If you then select 'Management' button, within Scrum Master You would see that competency on left.

Note the role associations

The screenshot shows the 'Essence Explorer' application interface. At the top, there's a navigation bar with 'File', 'Filter', 'Validate', and 'Help' menus, and a search bar. Below this is a 'Choose an Element:' section with radio buttons for 'Alpha', 'Competencies/Roles' (selected), 'Activity Spaces', 'Activities', 'Work Product', and 'Pattern'. A 'Collapse All' button is also present.

The main content area is divided into three horizontal sections: 'Customer', 'Solution', and 'Endeavour'. Each section contains several cards with star icons and labels. In the 'Endeavour' section, the 'Scrum' card is highlighted.

Below the main content area, there's a section for 'Alpha' with 'Choose Alpha' and 'Show All' buttons. Underneath, there's a 'Competency / Roles' section with a 'Choose Roles' button.

Two detailed cards are shown side-by-side. The left card is 'Stakeholder Representation' (marked with a star icon). It describes the competency as the ability to gather, communicate, and balance the needs of other stakeholders. It lists five sub-points: 1. Assists, 2. Applies, 3. Masters, 4. Adapts, 5. Innovates. The right card is 'Scrum Master' (marked with a person icon). It describes the role as accountable for ensuring Scrum is understood and enacted. It lists several responsibilities: Coaching self-management and cross-functionality, Removing impediments, Helping the Product Owner manage the Product Backlog effectively, Helping the Team focus and create high-value products, and Helping the organization understand and enact Scrum. It also lists three associated competencies: Management = 3, Scrum = 3, and Leadership = 3. A red arrow points from the 'Scrum Master' card to the 'Stakeholder Representation' card.

At the bottom of the 'Scrum Master' card, there's an 'Associations' section listing various roles and their associated competencies: Scrum Team - Part Of, Sprint Planning - Facilitates, Sprint Retrospective - Facilitates, Developers - Coaches and Helps, Product Owner - Coaches and Helps, Developers - Coaches and Helps, and Improvement - Drives Relentless.

Explore Activity Spaces, and the Contained Activities



Show the Activity Spaces and their contained Activities.

The 'Implement the System' Activity Space has the 'Accept a User Story' Activity.

If you select that then you can explore this Activity

The screenshot shows the 'Essence Explorer' application interface. At the top, there is a blue header bar with the title 'Essence Explorer' and navigation links: 'File', 'Filter', 'Validate', and 'Help'. A search bar is located on the right side of the header.

Below the header, a section titled 'Choose an Element:' contains radio buttons for 'Alpha', 'Competencies/Roles', 'Activity Spaces' (which is selected), 'Activities', 'Work Product', and 'Pattern'. A 'Collapse All' button is also present.

The main content area is divided into three horizontal sections, each with a title and a set of buttons:

- Customer**: Buttons include 'Explore Possibilities', 'Understand Stakeholder Needs', 'Ensure Stakeholder Satisfaction', and 'Use the System'.
- Solution**: Buttons include 'Understand the Requirements', 'Shape the system', 'Implement the System', 'Test the System', 'Deploy the System', and 'Operate the System'.
- Endeavour**: Buttons include 'Prepare to do the Work', 'Coordinate Activity', 'Support the Team', 'Track Progress', and 'Stop the Work'.

Below these sections, a sidebar on the left lists the following elements with expand/collapse arrows:

- Alpha
- Competency / Roles
- Activity Space (expanded)

The expanded 'Activity Space' section shows a detailed view of the 'Implement the System' activity space. It includes a description: 'Build a system by implementing, testing, and integrating one or more system elements. This includes bug fixing and unit testing'. It also lists 'Implement the system to:' with a bulleted list of tasks: 'Create a working system', 'Develop, integrate, and test the system elements', 'Increase the number of requirements implemented', 'Fix defects', and 'Improve the system'.

Below the description, there are three sections:

- Input**: 'Requirements, Software System, Way of Working'
- Entry**: 'Software System::Architecture Selected'
- Completion**: 'Software System::Ready'

A large red arrow points from the 'Implement the System' activity space to the 'Activities' section on the right. The 'Activities' section shows a single activity: 'Accept a User Story'.

Explore Activities – Choose ‘Product Backlog Refinement’ in Concerns



Choose an Element:

Alpha Competencies/Roles Activity Spaces **Activities** Work Product Pattern [Collapse All](#) ☒ Single View

Customer

Solution

☐ Find User Stories ☐ Prepare a User Story ☐ Accept a User Story ☐ **Product Backlog Refinement**

Endeavour

☐ Daily Scrum ☐ Sprint Planning ☐ Sprint Retrospective ☐ Sprint Review

Alpha

Competency / Roles

Activity Space

Activity

[Choose Activity](#)

Product Backlog Refinement

The ongoing activity of breaking down, and further defining Product Backlog Items into smaller, more precise items (adding detail such as description, order and size). A whole team activity led by the Product Owner

Entry

Competencies and Roles

Competencies: Leadership = 3, Stakeholder Representation = 3

Roles: Product Owner, Scrum Team

Achieves

- update - Requirements to: Conceived or beyond (mandatory)
- update - Requirements to: Conceived (mandatory)
- update - Product Backlog to: Goal Set or beyond (mandatory)
- update - Product Backlog to: Goal Set (mandatory)
- update - Product Backlog Item to: Identified (mandatory)
- update - Product Backlog Item to: Ready (mandatory)
- update - Product Goal to: Identified or beyond (mandatory)
- update - Product Goal to: Identified (mandatory)

Contributes To

Patterns

Associations

- Part Of: Understand the Requirements

Elements Impacted

☐ Requirements

☐ Product Backlog

☒ Product Backlog Item

☒ Product Backlog Item

☒ Product Goal



Select and
browse the
Elements
actioned by
Activities

Search for 'test case' Work Product



Choose to Filter the Practices as User Story and Scrum, exclude Kernel. Enter 'test case' in search box. Press Search

Essence ExplorerFileFilterValidateHelp

Choose an Element

✓ User Story

✓ Scrum

Kernel

Alpha

Competencies/Practices

Activities

Work Product

Pattern

Collapse All

Customer

Solution

Story Card

Test Case

Definition of Done

Sprint Backlog

Increment

Product Backlog

Endeavour

Alpha

Competency / Roles

Activity Space

Activity

Work Product

Choose Work Product

Test Case

Test Ideas Captured

Executable

Automated

Describes Alpha: User Story

A set of actions to be performed on the solution to determine if the conditions of satisfaction are met correctly. There is often more than one Test Case for each User Story. Test Cases include acceptance tests (which the Customer Team is responsible for) plus additional tests defined by the developers.

Test Case

Test Ideas Captured

Justification that the test case has sufficient rigour and completeness to give confidence that the story has been completed satisfactorily

Mandatory

- The key things that need to be tested have been documented
- The Customer Team is happy that the test ideas together verify an acceptable implementation of the User Story

Test Case is found. 'Test Ideas Captured' Details shown on the Right with its CheckBoxes.

Elements can have **resource** documents and 'Artefacts' associated with them: HTML pages, PDFs, Images etc.

These Elements have an 'Artefact' or 'Detail' button.

A 'SOAP template' artefact is shown below the SOAP Work Product.

SOAP is an acronym for "Solution On A Page". It is an early summary of a solution, used to gain Stakeholder agreement.

Detail documents elaborate on Element Cards summarized text.

Work Product

Choose Work Product

SOAP - Solution on a page

High Level

A Solution On a Page is an early, or high level, description of the problem and the initiative with its potential solution described, the stakeholders, the solution context, business drivers, benefits and outcomes.

In some cases, a SOAP may be all that's required, as an alternative to a SAD or Solution Blueprint.

Artefact

Describes Alpha: User Story

Close Artefact

SOAP - Solution on a page

High Level

Describe the system context, and predominantly identify stakeholders, and the extent of the black box solution that provides the basis for requirements gathering. Define the technical complexity

Optional

- Identify Business Outcomes
- System Context and Interfaces
- Technical Complexity defined

Architecture

Solution-on-a-Page : INITIATIVE TITLE <IDs if available>

Version 0.2 01 March 2022 Pg 1

Drivers & Benefits

The business problem as a business driver - for instance, move current product to new market. Ideally this reflects one or more value and operational drivers

Provide an extended market for Service technicians for Service suppliers. Providing more accurate parts knowledge for Technicians

The benefit is a new market base.

Business Outcomes

The business outcome delivered by the solution in terms as independent as possible of IT solution - really a description of the business solution.

Business scenario XYZ will be providing

Provide a B2B API search capability to the Service provider with accurate parts matching to Service ID and parts pricing.

IT Solution Description

The IT solution in words, describing the picture to the right..

A technician will use a customer application who's system connects our Service API via NGINX gateway through to Service System. Gateway records usage, and request-response times and throttles if over limit.

The system request provides a Service ID and the correct parts are returned with a price. Records loaded into Tableau for reporting. Billing based on only records within time SLA. Billing report with only SLA met usage

System Impacts

Component	Impact	Details
Auth	10%	Additional AAA Entries/config
NGINX Proxy	40%	Upgrade to NGINX Plus (+Throttling)
Service System	10%	Restrict current API to required fields
Data	20%	New Data for Service Items
Tableau	10%	New billing codes Usage reporting, monitoring

Utilise this slide template for all context & roadmap solution on a page capture.

- Highlight picture call-outs with circles of accent 3 (gold) and no outline, white text as shown adjacent. Reference these within the text using windings numbers e.g. 1
- On the solution component diagram, show unimpacted components that are relevant to the architecture in white darker 25%.
- Show impacted components in Accent 1
- Show unidentified components in Accent 3

Interfaces

- Unimpacted but required
- Impacted
- New

Top 3 System Qualities

Quality	Requirements
Security	API Authorisation, SSL
Availability	Throttle to limit of 5,000 per hr.
Performance	2 seconds max per request

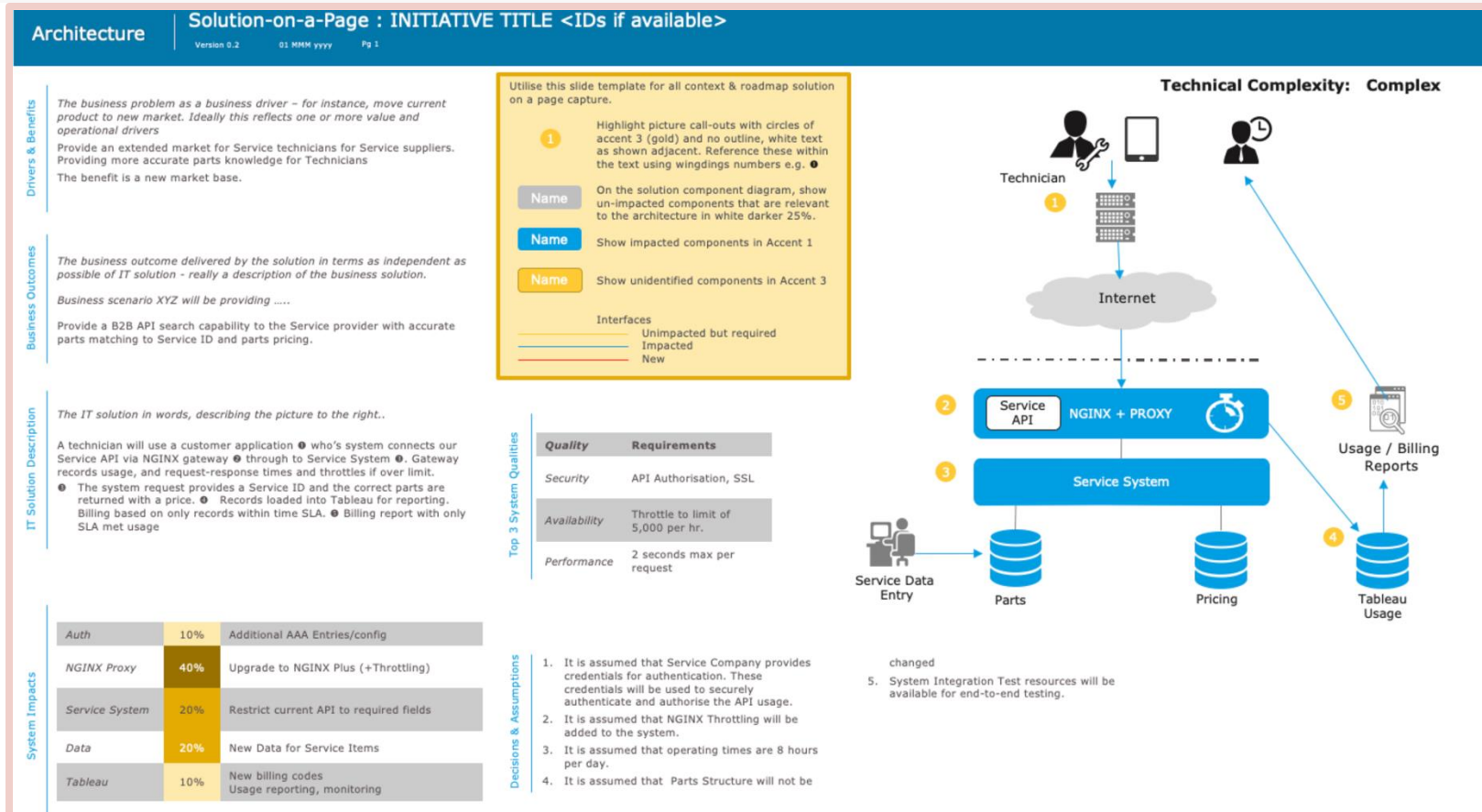
Decisions & Assumptions

- It is assumed that Service Company provides credentials for authentication. These credentials will be used to securely authenticate and authorise the API usage.
- It is assumed that NGINX Throttling will be added to the system.
- It is assumed that operating times are 8 hours per day.
- It is assumed that Parts Structure will not be changed
- System Integration Test resources will be available for end-to-end testing.

Technical Complexity: Complex



A SOAP Artefact – Resource Detail



Patterns – Example 3 Scrum patterns



The single view toggle allows a single result to be viewed. Toggling this allows multiple accordions to be opened, as required, to view across elements

Essence Explorer

FileFilterValidateHelp

Search

Search

Choose an Element:

Alpha

Competencies/Roles

Activity Spaces

Activities

Work Product

Pattern

Collapse All

Single View

Customer

Solution

Endeavour

Self Management

Transparency

Cross Functional Team

Alpha

Competency / Roles

Activity Space

Activity

Work Product

Pattern

Self Management

Self-managing teams are teams that internally decide who does what, when and how

Type: advisory

Associations

- Scrum Team - Practice



Practice

Choose Practice ▾

Kernel
Scrum
User Story

Kernel

Opportunity

α Stakeholders

α Requirements

α Software System

α Work

α Team

Competency

☆ Stakeholder Representation

☆ Analysis

☆ Development

☆ Testing

☆ Leadership

☆ Management

Role

Activity Space

🔲 Explore Possibilities

🔲 Understand Stakeholder Needs

🔲 Ensure Stakeholder Satisfaction

🔲 Use the System

🔲 Understand the Requirements

🔲 Shape the system

🔲 Implement the System

🔲 Test the System

🔲 Deploy the System

🔲 Operate the System

🔲 Prepare to do the Work

🔲 Coordinate Activity

🔲 Support the Team

🔲 Track Progress

🔲 Stop the Work

Activity

Work Product

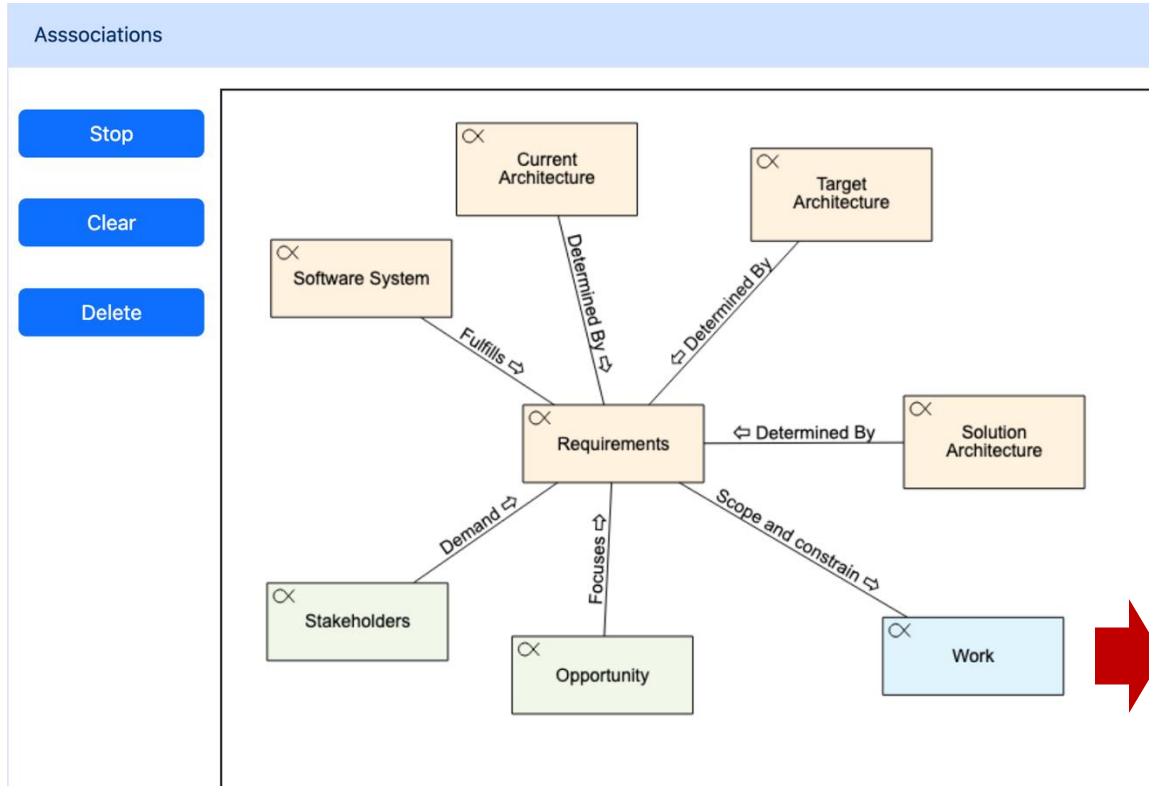
Pattern



Associations and relationships are an important part of Essence. An Element, such as Requirements, can be selected then The associations displayed and explored.

Exploring is also driven by selecting elements from the Concern Window:

- **Stop** - stops exploring and freezes the diagram
- **Clear** - allows you to start a new diagram
- **Delete** - removes a selected element and its associations
- **Selected Elements** can be moved around the canvas, and resized.
- **Double click** takes you to the cards as in prior slides.
- Relationship directions ' $\Rightarrow$ ' is shown.
- An Element can be further explored, see next page, where 'Work' is followed.



Association Chain Further Explored

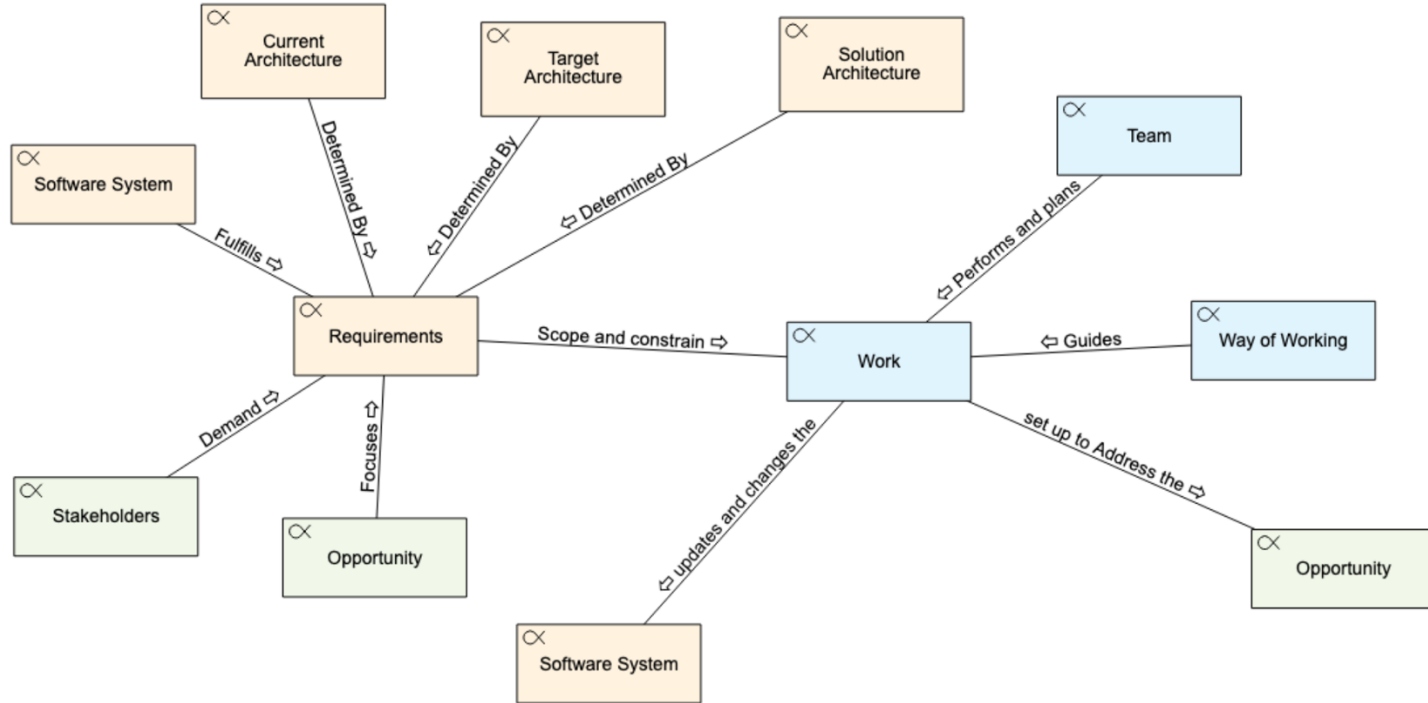


Associations

Stop

Clear

Delete



Discover the Kernel Relationships (as per spec, see slide 15)

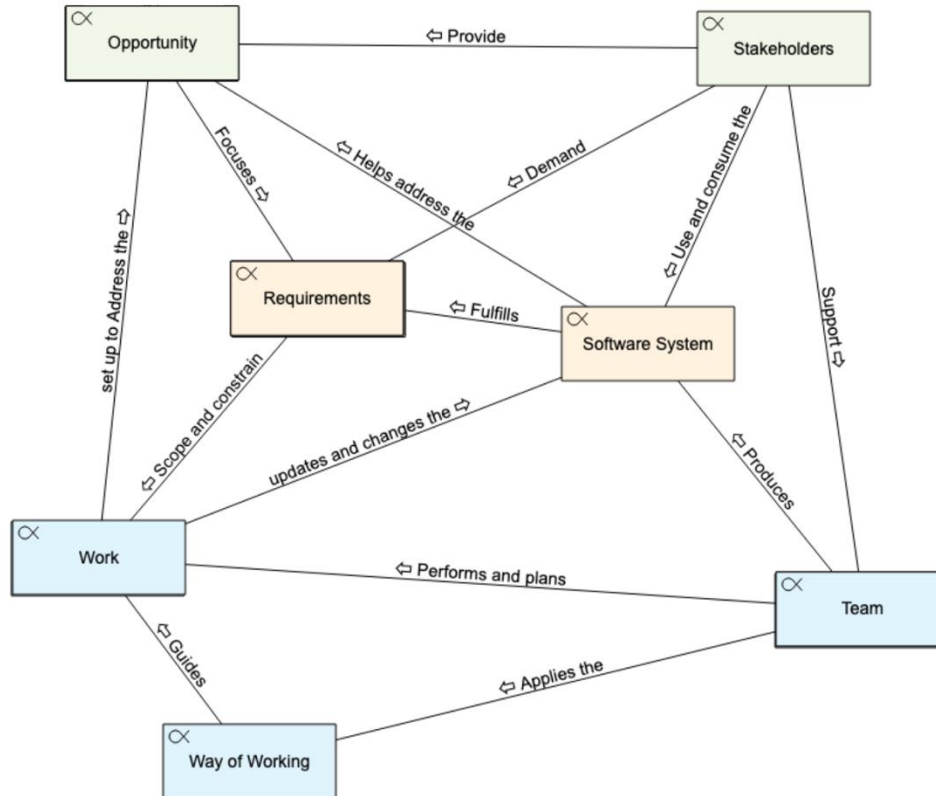


Associations

Stop

Clear

Delete



Customer

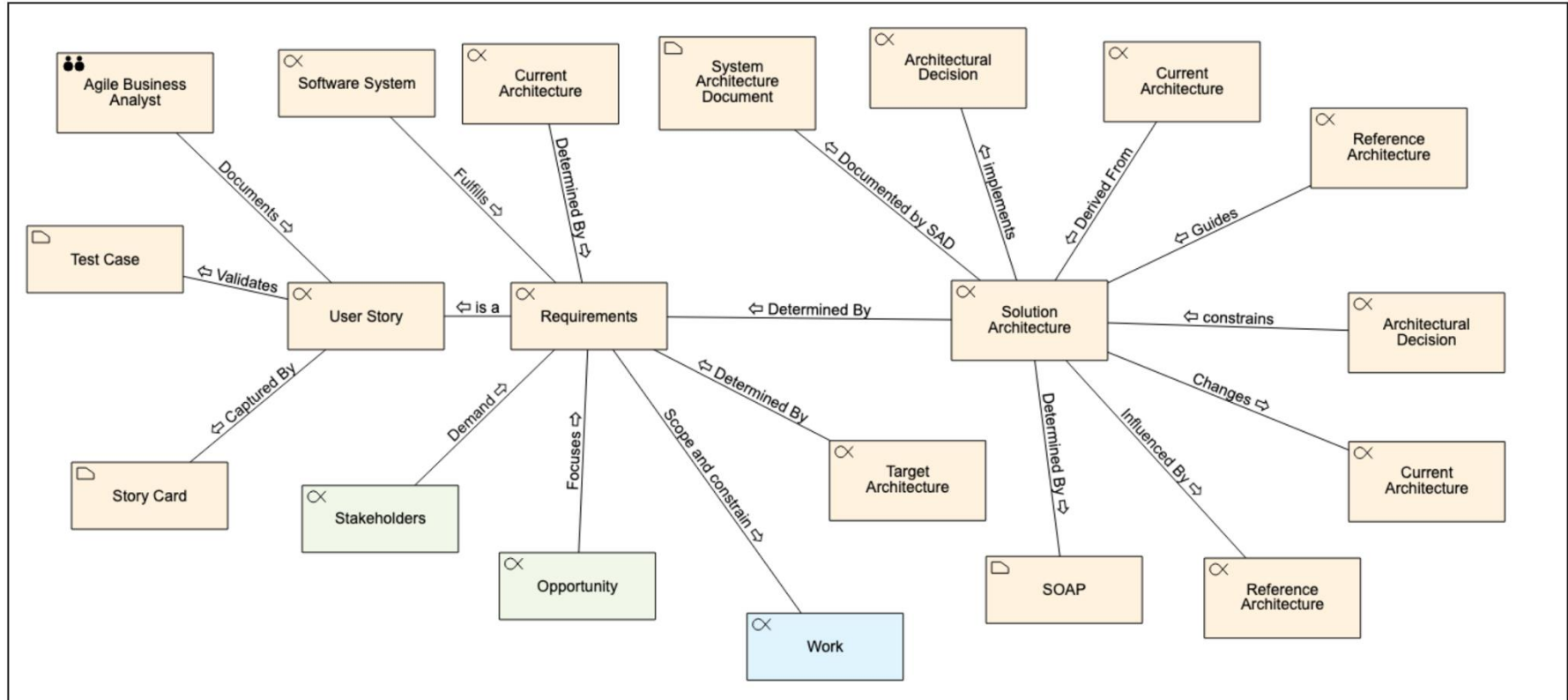
Solution

Endeavour

Explore Alpha Inheritance Relationships and Element Types

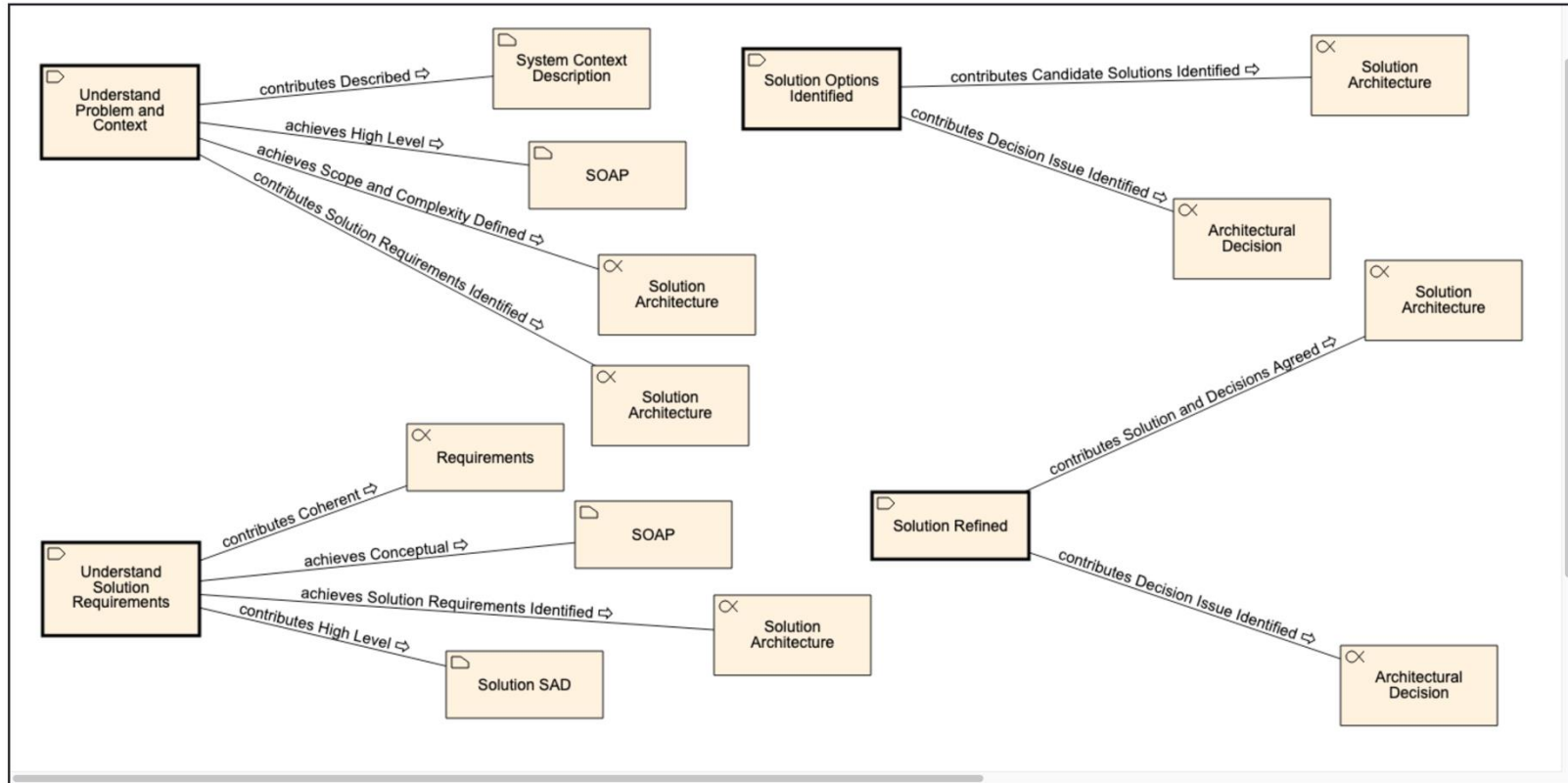


The 'User Story' alpha inherits, 'is a', from the kernel 'Requirements' alpha. The 'Solution Architecture' is 'Determined By' 'Requirements'. Any element type can be displayed, e.g., Roles, Alpha, and Work Products, etc.





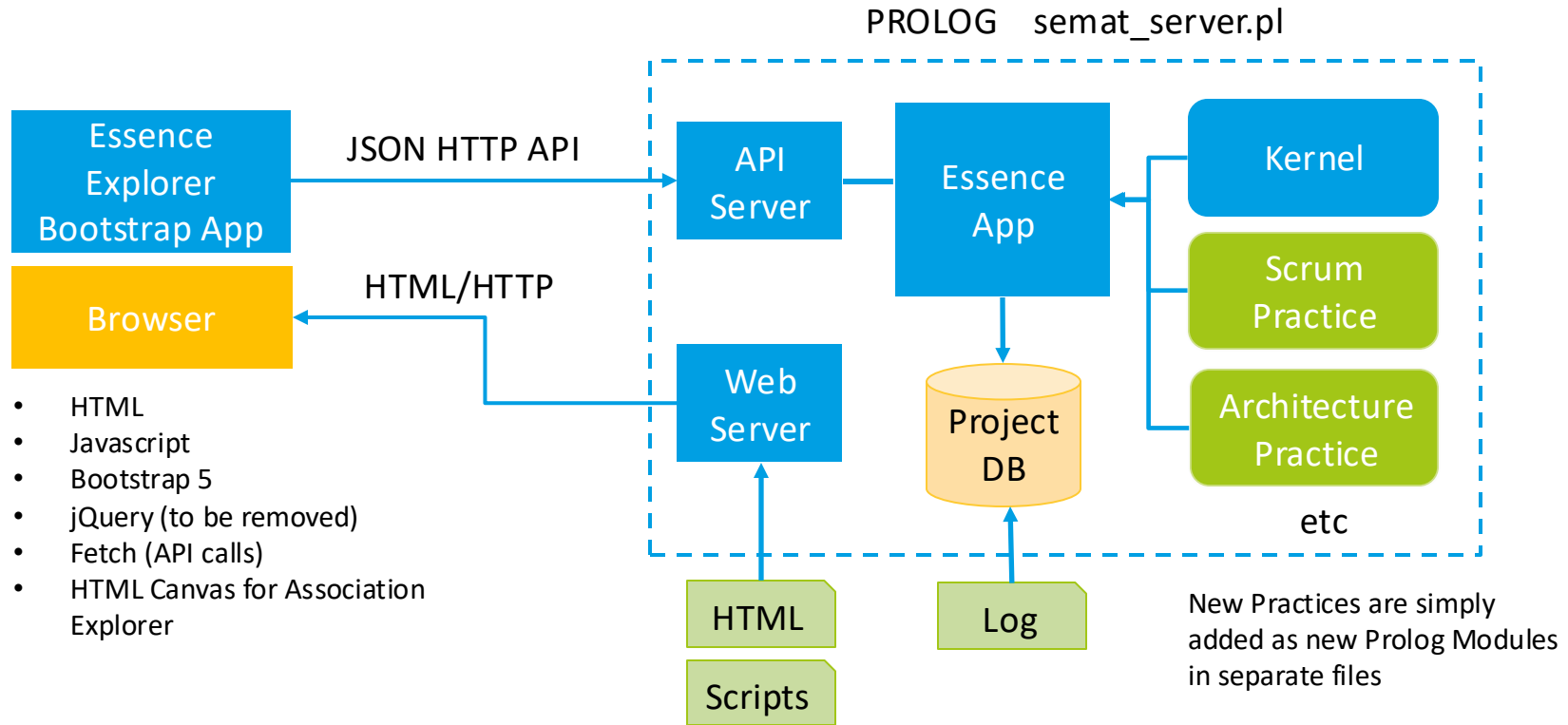
The links show the actions on the target elements from 4 architecture related activities





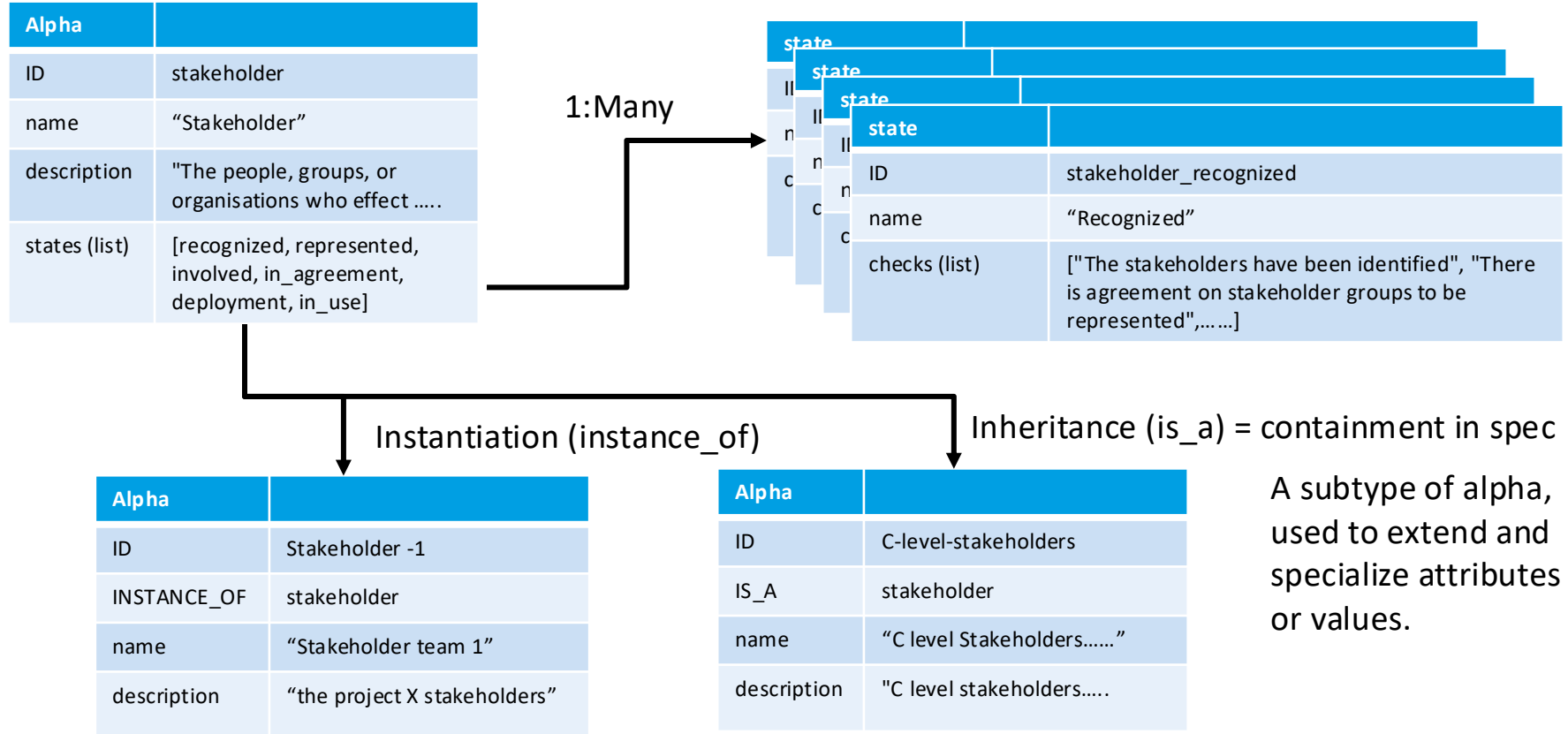
Prolog and API

Client-Server Architecture – MVC based.



Dynamic Pages: Essence changes in the Prolog data are immediately reflected in the front end Explorer App. Editing is interactive. The Server does not generate HTML but serves data to the front end app.

Essence Entities are represented, in Prolog, as OAV Typed Objects.



A subtype of alpha, used to extend and specialize attributes or values.

Activity Space – Activity - Action



Activity_Space	
ID	coordinate_activity
name	"Coordinate Activity"
description	["Co-ordinate and direct the team's work....."]
prompt	"Coordinate activity to:"
points	["Select and prioritize work", "Adapt plans to"]
input	[requirements, team, work, wow]
entry	[team:seeded, work:prepared]
completion	[team:formed, work:under_control]

Activity	
ID	sprint_planning
PART_OF	coordinate_activity
name	"Sprint Planning"
description	["Collaboratively plan the work to be perform...."]
approach	
competencies	[leadership at_level 3, management at_level 2]
roles	[scrum_master, scrum_team]
patterns	[....]
entry criteria	
completion	

PART_OF

Activities can only be started when their Activity Space Entry Criteria are met

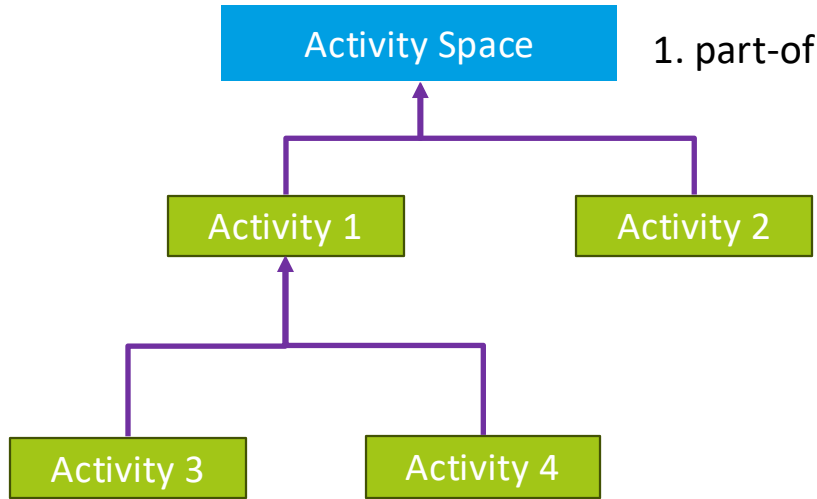
Action	
ID	sprint_planning
...	

An Activity has many Actions



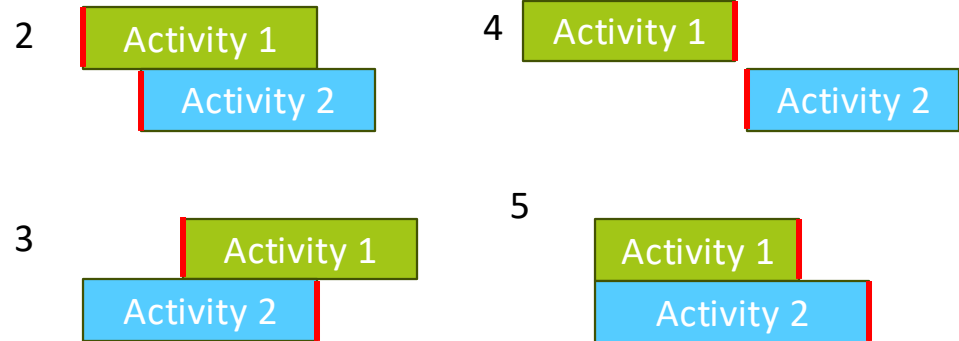
- **Create** - The activity creates the work product. It is likely to use this kind of operation in activities that set up an environment or create initial version of work products.
- **Read** - The activity reads the work product but does not change it. This kind of operation assumes that the work product needs to be present to be successful in this activity.
 - It is likely to use this kind of operation in activities that transform contents from one work product into other work products.
- **Update** - The activity possibly modifies the work product. In an actual endeavour, there may be cases in which no modification is necessary, but there is at least one case in which the work product has changed after performing the activity.
 - This kind of operation assumes that the work product needs to be present to be successful in this activity.
- **Delete** - The activity deletes the work product. This kind of operation assumes that the work product no longer exists if the activity is completed successfully. Note that deleted work products cannot be covered by completion criteria.
 - It is likely to use this kind of operation in activities that finalize an endeavor and thus remove intermediate results for privacy or security reasons.

Activity Associations – Work Breakdown Structure



Relationships:

1. part-of
2. start-before-start
3. start-before-end
4. end-before-start
5. end-before-end



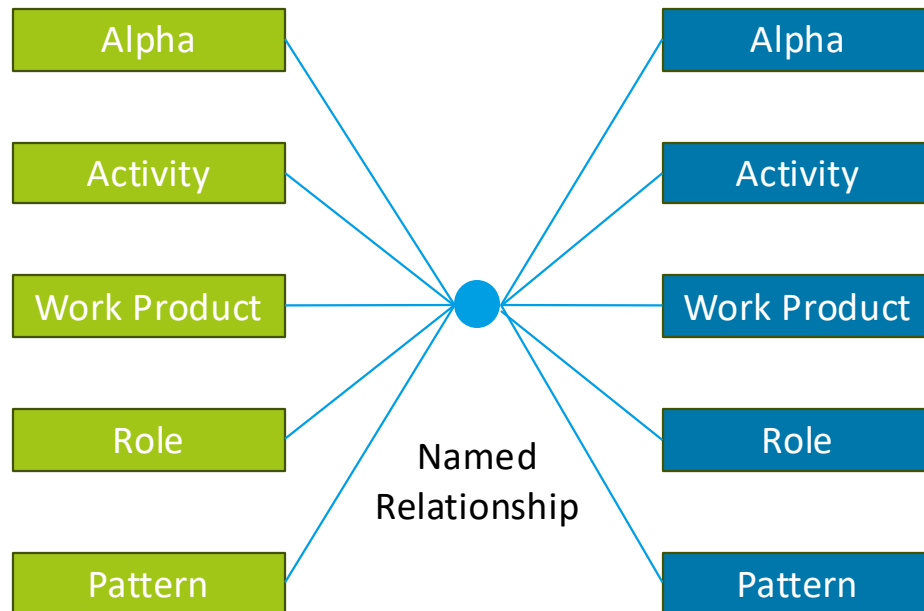
Example Prolog showing relationship between 2 activities:

```
activity_association(end_before_start, find_user_story, prepare_user_story ).  
activity_association(end_before_start, prepare_user_story, accept_user_story ).
```

Associations can be specified between most element types



Examples



An Improvement is Added to the Sprint Backlog

Developers are Responsible for Conducting the Daily Scrum

Sprint Backlog Includes an Improvement

Product Owner is Part Of Scrum Team

A Scrum team Is Cross Functional

Prolog Examples:

```
association("Added to", improvement, sprint_backlog).  
association("Focuses", opportunity, requirements).  
association("Scope and constrain", requirements, work).
```

Roles and Competencies



The Essence Standard, models Roles as Patterns. However, Roles can be 1st class Entities that are composed of competencies at specified levels; the theory under test

Role	
ID	scrum_master
name	"Scrum Master"
description	"The Scrum Master is accountable for ensuring...."
competencies	[management at_level 3, scrum at_level 3, leadership at_level 3]
points	"They are true leaders who serve the team in several ways" items ["Coaching self-management and cross-functionality", "Removing impediments", "Helping the Product Owner manage the Product Backlog effectively",} at_level 1

has

Competency	
ID	leadership
name	Competency
description	ID management
states (list)	name "Management"
	description "This competency encapsulates the ability to coordinate, plan and track the work done by a team....."
	states (list) [assists, applies, masters, adapts, innovates]

Associations

- Part Of - Scrum Team
- Facilitates - Sprint Planning
- Facilitates - Sprint Retrospective
- Coaches and Helps - Developers
- Coaches and Helps - Product Owner
- Drives Relentless - Improvement
- Part Of - Scrum Team

A scrum master has competencies:

- Management at Masters level (3)
- Scrum at Masters level (3)
- Leadership at Masters level (3)

Example REST API* calls to get an Alpha, and States



A list of all Elements can be obtained, that provides the IDs for further queries e.g., get all alphas.

GET: `http://localhost:8000/api/elements/alpha`

Returns the list of 'alpha' IDs, e.g. team, work, solution....

Now we can get the a particular 'alpha' with the ID.

GET: `http://localhost:8000/api/element/team`

Response



Alpha have states as shown on right. Each State has an ID that is then used in separate call to get the State details, e.g. get team seeded details:

GET: `http://localhost:8000/api/element/team/detail/seeded`

```
{
  "id": "team",
  "concern": "endeavour",
  "title": "Team",
  "summary": "The group of people actively engaged in the development,
    maintenance, delivery and support of a specific software
system.",
  "states": [
    "seeded",
    "formed",
    "collaborating",
    "performing",
    "adjourned"
  ],
  "names": [
    "Seeded",
    "Formed",
    "Collaborating",
    "Performing",
    "Adjourned"
  ],
  "items": [
    "A healthy Team meets its team goals effectively",
    "A healthy Team has members that collaborates effectively",
    "A healthy Team focus on their work",
    "A healthy Team continually improves"
  ],
  "associations": [
    "Way of Working - Team applies the way of working",
    "Work - Team performs and plans work ",
    "Software System - Team produces software system"
  ]
}
```



Two Approaches:

1) Edit the raw Prolog Code (allows Coding tool, e.g. Visual Studio, to do full refactoring easily):

```
alpha(wow, name, "Way of Working").  
alpha(wow, description, "The tailored set of practices and .....").  
alpha(wow, points, ["Good way of working is agreed by the team ", "Good way of ....."]).  
alpha(wow, states, [principles_established, foundation_established, in_use, in_place, working_well]).
```

2) Prolog UI Helper to enter new Elements, these generate the Prolog code:

```
Enter Alpha Name: Way of Working  
Enter Description: The tailored set of practices and ....  
Enter Points 1: Good way of working is agreed by the team  
Enter Points 2: Good way of working reduces risks and technical debt  
...  
...  
Enter State: etc etc..  
Enter Check Point: etc etc
```

- Both above can be loaded, and if no Prolog errors, use the Validate Tools to check all is good.
- **Edinburgh** standard Prolog is used for all the Essence data, but SWIPL addons, e.g. Meta predicates, Lambda expressions are used for the Web APIs. So other Prologs can use the data parts of this work, as is.

Using Domain Specific Languages (DSLs) in Prolog code



```
activity(sprint_retrospective, name, "Sprint Retrospective").  
activity(sprint_retrospective, description, "Collaboratively plan and enact ways to increase quality  
and effectiveness. A Scrum Team event of no more than 3 hours").  
activity(sprint_retrospective, competencies, [Leadership at Level 3, management at Level 2]).  
  
action(sprint_retrospective, optional, update, to_achieve sprint_backlog to_detail primed).  
action(sprint_retrospective, mandatory, update, contributes_to sprint to_state reviewed or beyond).  
...etc, etc.
```

The ‘**sprint_retrospective**’ is the key for the 5 example facts above.

The **competencies** required are:

- Leadership at level 3
- Management at level 2

The **actions DSL** can be read as:

- Optionally update a Work Product, to achieve sprint backlog to detail level primed.
- Mandatory update an Alpha, contribute to the sprint to state reviewed or beyond.



AI Research

Creating An Essence Explorer Chat Bot



The Bot uses an LLM, Chat-GPT, to provide an Expert Assistant in Essence that knows about all the information in the Essence Explorer knowledge base. It knows all the tailored information, about the practices, developed by the user in the Explorer.

LLMs understand English better than Prolog, and so the first step is to convert the Essence Explorer Prolog Data to English, and extract this into a set of documents. This data is based on the Explorer user's developed Practices.

- Extracts other than English can be created, e.g. SQL, Java, Python etc.
- Another research extract is a Neo4j Cypher Knowledge Graph. This can be used to explore Essence as a Knowledge Graph.

A set of these documents, one for each practice is then provided, in a vector store, and a RAG approach is used for the LLM.

- A document describing the background to Essence is also supplied. It has definitions for what Essence is and what *Practices*, *Concerns*, *Alphas*, *Work Products*, *Activities*, etc. are. This taken from the OMG standard.
- In the first step of a RAG the Question is matched against the documents, and only the parts that match are sent to the LLM. All the documents are not sent to the LLM, reducing the cost of tokenization with OpenAI.
- **The core to this approach is the Essence Explorer Prolog Data, its declarative approach, allowing rich documents ready for AI, that are based on well defined relationships between all the Essence information, to be easily created.**
- This data grounds the LLM, so it does not make silly statements about Essence (hallucinations), and it provides all the information required to be an expert assistant based on the core and users' practices.

The following example shows how we can ask questions about Essence, the Kernel and for example the Scrum Practice.



An example English Extract of the Kernel, from Prolog, showing some of the “Requirements” Alpha:

Requirements is an Alpha in the practice named ‘kernel’ for the concern called ‘solution’.

Requirements can be described as: What the software system must do to address the opportunity and satisfy the stakeholders.

Requirements Features include:

- Good Requirements meets real needs,
- Good Requirements have clear scope,
- Good Requirements are coherent and well organized,
- Good Requirements help drive development,

Requirements go through the following states: Conceived, Bounded, Coherent, Acceptable, Addressed, Fulfilled, Each of these is described below.

Requirements state Conceived has these checkpoints:

- The need for a new system is clear,
- Users are identified,
- Initial sponsors are identified.

Requirements state Bounded has these checkpoints:

- The purpose and extent of the system are agreed ,
- Success criteria are clear ,
- Mechanisms for handling requirements are agreed ,
- Constraints and assumptions identified.

Requirements state Coherent has these checkpoints:

- The big picture is clear and shared by all involved,
- Important usage scenarios explained,
- Priorities are clear,

.....

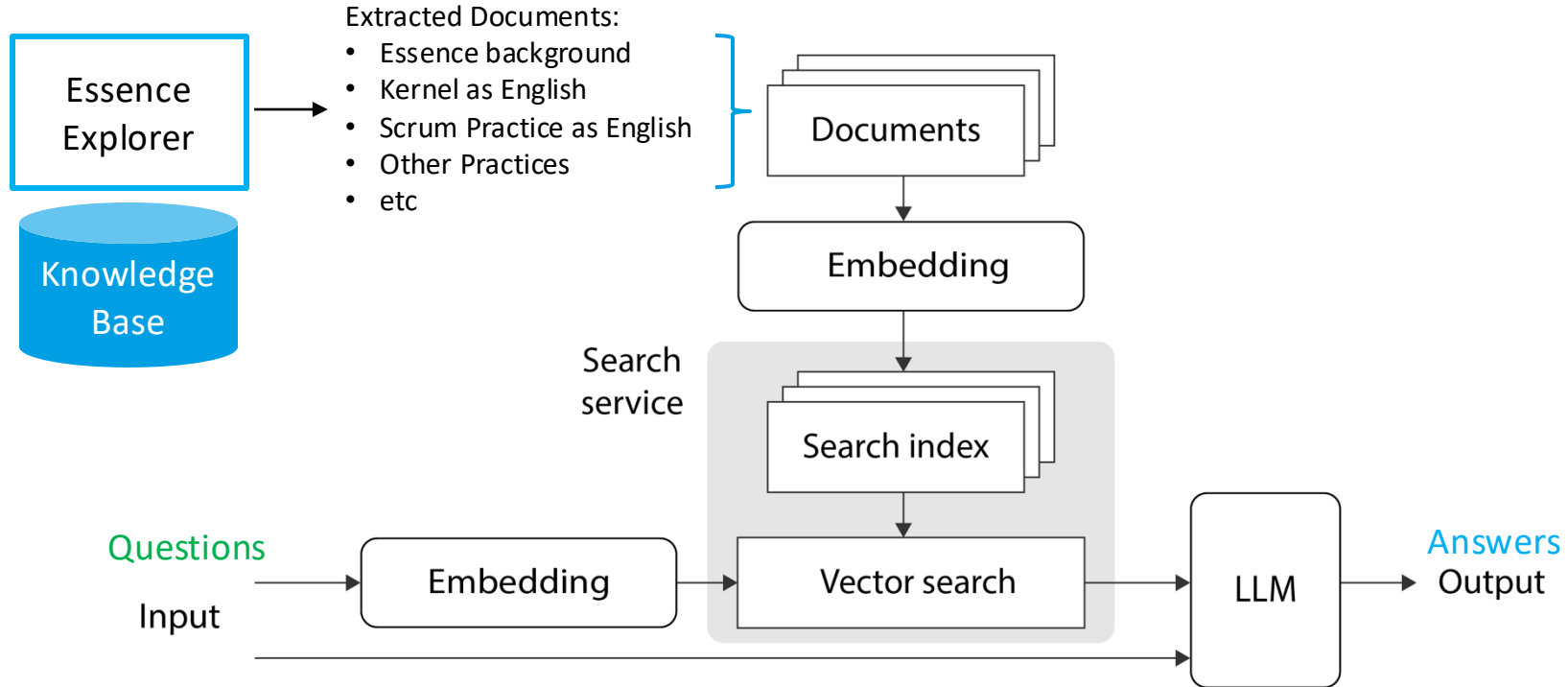
.....

Each Practice can be extracted in an English Document, with descriptions of all the Alphas, States, Work Products, Activities, Activity Spaces, Associations etc.

RAG Approach Architecture – Creating a Chat Bot



Retrieval Augmented Generation (RAG) is an approach to ground and extend an LLM, with custom data, documents, that provide specific knowledge. In this case, the documents cover Essence and specific practices.



Now let's Chat with Essence Explorer - Part 1



Question: What is the Essence kernel?

Answer: The Essence Kernel is a foundational framework that captures the essential elements of software engineering that are integral to all software engineering methods. It provides a common ground for defining software development practices and includes elements such as Requirements, Software System, Team, and Work. The Kernel helps practitioners compare methods and make better decisions about their practices. It is described using the Essence Language, which defines abstract syntax, dynamic semantics, graphic syntax, and textual syntax.

Question: What are the areas on concern ?

Answer: The Kernel is organized into three areas of concern:

1. Customer
2. Solution
3. Endeavor

Question: What is a Method?

Answer: A Method is a composition of practices in software engineering. It is not just a description for developers to read but a dynamic tool that supports their day-to-day activities. A method describes what is done in a software engineering endeavour, rather than just what is expected to be done.

Question: What is an endeavour?

Answer: In the context of the Essence Kernel, an endeavour is an area of concern that contains everything to do with the team and the way they approach their work. It involves forming the team, defining their way of working, and carrying out the work needed to develop, maintain, and support a software system. Essentially, the endeavour area of concern focuses on the team and their approach to software engineering tasks.

Now let's Chat with Essence Explorer - Part 2



Question: What is an Alpha ?

Answer: An Alpha is an essential element that captures key concepts involved in software engineering. It allows the progress and health of a software engineering endeavour to be tracked and assessed. Alphas provide common ground for defining software engineering methods and practices. Each Alpha has a small set of predefined states with associated checklists to assess progress and health. The states of Alphas are not just linear progressions and can be iterated through based on the team's choice of practices.

Question: What is a requirement ?

Answer: A requirement can be described as what the software system must do to address the opportunity and satisfy the stakeholders. Good requirements meet real needs, have a clear scope, are coherent and well-organized, and help drive development. Requirements go through states such as Conceived, Bounded, Coherent, Acceptable, Addressed, and Fulfilled. Each state has specific checkpoints that need to be met to progress to the next state.

Question: Is a Requirement an Alpha?

Answer: Yes, a Requirement is an Alpha.

Question: What are the Requirements associations ?

Answer: The associations of Requirements are:

- Requirements Scope and constrain Work
- Opportunity Focuses Requirements
- Stakeholders Demand Requirements
- Software System Fulfills Requirements
- Current Architecture Determined By Requirements
- Target Architecture Determined By Requirements
- Solution Architecture Determined By Requirements

Now let's Chat with Essence Explorer - Part 3



Question: What are states of requirement ?

Answer: The states of Requirements are:

1. Conceived
2. Bounded
3. Coherent
4. Acceptable
5. Addressed
6. Fulfilled

Question: What is a Requirement conceived state ?

Answer: In the Requirements practice, the "Conceived" state is the initial stage of the Requirements Alpha.

The checkpoints for the Requirements state Conceived include:

- The need for a new system is clear,
- Users are identified,
- Initial sponsors are identified.

This state signifies the early identification of the need for a new system and the key stakeholders involved in the project.

Question: what is a checkpoint?

Answer: A checkpoint is a specific criteria or condition that must be met or validated in order to progress to the next stage or state in a process or framework. Checkpoints are used to ensure that certain requirements or expectations are fulfilled before moving forward.

Question: What state comes after conceived ?

Answer: The state that comes after "Conceived" for Requirements is "Bounded".

Now let's Chat with Essence Explorer - Part 4



Question: Tell me the checkpoints for Bounded?

Answer: Requirements state Bounded has the following checkpoints:

- The purpose and extent of the system are agreed,
- Success criteria are clear,
- Mechanisms for handling requirements are agreed,
- Constraints and assumptions identified.

Question: What is a work product ?

Answer: A work product is an artefact of value and relevance for a software engineering endeavor. It can be a document, a piece of software, or other created entities such as the creation of a test environment or the delivery of a training course.

Question: What is a sprint ?

Answer: A sprint is a time-box of one month or less in the practice of 'scrum' for the concern called 'endeavour'. It is described as the heartbeat of Scrum, where ideas are turned into value. A new Sprint starts immediately after the conclusion of the previous Sprint, and regular Sprints create consistency and enable predictability. Sprint includes Scheduled, Planned, and Reviewed states, each with their own specific checkpoints.

Question: What is the concern for a Sprint?

Answer: The concern in which a Sprint is associated is called "endeavour."

Question: What are a Sprint associations ?"

Answer: The associations of a Sprint are:

- Sprint Creates one or more Increment
- Sprint Progresses Product Backlog
- Sprint Addresses Sprint Goal.....

Now let's Chat with Essence Explorer - Part 5



Question: What is an Increment ?

Answer: An increment is a concrete stepping stone towards the Product Goal. A Sprint may produce multiple Increments. Each Increment is additive to all prior Increments. In order to provide value the Increment must be usable and meet the Definition of Done.

Question: Is an Increment a work product?

Answer: Yes, an Increment is a Work Product.

Question: Describe a Work product?

Answer: A work product is an artefact that holds value and relevance for a software engineering endeavour. It can take the form of a document, a piece of software, or other created entities such as the creation of a test environment or the delivery of a training course. Work products are essential elements in the software development process, contributing to the progress and success of the endeavour. They are tangible outputs that result from the activities performed during the software engineering process.

Question: Tell me about activities?

Answer: In the Essence Software Engineering Framework, an Activity is defined as an approach for carrying out work to be performed. Activities recommend actions on alphas and/or work products to help perform the work effectively. Activities provide guidance on how to progress through the different states of the alphas and how to create or modify work products to achieve the desired outcomes. Each activity is designed to support the team in successfully completing the tasks required to develop, maintain, and support a software system.

Question: what are the Alphas in scrum

Answer: The Alphas in Scrum are: 1) Sprint 2) Improvement 3) Sprint Goal 4) Product Goal 5) Product Backlog Item

Question: what are the Alphas in kernel

Answer: The Alphas in the kernel are: 1) Software System 2) Stakeholders 3) Opportunity 4) Requirements 5) Team 6) Work 7) Way of Working



Essence Chat

Question: what is Essence

Answer: Essence is a software engineering framework that consists of the Essence Kernel and the Essence Language. The Essence Kernel provides the common ground for defining software development practices, while the Essence Language is a domain-specific language used to define methods, practices, and kernels within the framework. Together, they separate the "what" of software engineering (Kernel) from the "how" (practices and methods), providing a common vocabulary for discussing software engineering and a foundation for defining practices and methods. The Essence framework aims to support practitioners and method engineers in improving decision-making, method use, and method building in software engineering endeavors.

Question: what is a current architecture

Answer: The Current Architecture describes the current state of an application, system, or platform. It includes emergent design, intentional design, SOAPs, and Solution Blueprints. It plays a critical role in explaining the existing system, the choices made, and how new development can be more agile. The Current Architecture is continually monitored and reviewed for changes as it evolves, capturing new intentional solutions, bug fixes, and emergent design.

Question: tell me about scrum

Answer: Scrum is a competency in the software engineering framework named 'scrum' for the concern called 'endeavour'. It encapsulates the knowledge and practice of scrum. In the context of scrum, there are specific roles involved: 1. Product Owner: Accountable for maximizing the value of the product resulting from the work of the Scrum Team 2. Developers: Committed to creating any aspect of a usable Increment each Sprint 3. Scrum Master: Accountable for ensuring that Scrum is understood and enacted The Scrum Team consists of the Product Owner, Developers, and a Scrum Master. They work together in small, focused teams of typically 10 or fewer people. Scrum operates in time-boxed iterations called Sprints, which are typically one month or less. The Sprint is the heartbeat of Scrum, where ideas are turned into value. Regular Sprints create consistency and enable predictability. During a Sprint, the Scrum Team works on creating Increments, which are concrete stepping stones towards the Product Goal. Each Sprint may produce multiple Increments that are additive to all prior Increments. In addition to Sprints, Scrum involves activities such as Daily Scrum, Sprint Planning, Sprint Review, Sprint Retrospective, and Product Backlog Refinement to optimize progress towards the Sprint Goal and improve the way the Scrum Team works. Overall, Scrum is a framework that emphasizes collaboration, adaptability, and delivering value in an iterative and incremental manner.

Question: what are the states of a current architecture

Answer: The states of a Current Architecture are: 1. Changes Identified 2. New or Modified Components Described 3. Models and Views Updated 4. Accepted

Question: tell me about a user story

Answer: A user story is a tool used in software development to capture a description of a software feature from an end-user perspective. It typically follows a simple template: "As a [role], I want [feature], so that [benefit]." User stories are used in Agile development methodologies to define the requirements and functionalities of a software system in a way that is easy to understand and prioritize. They help teams stay focused on delivering value to the end-users.

Question: is a user story produced in a sprint ?

Answer: In the Scrum framework, user stories are typically part of the Product Backlog, which is an ordered list of items needed to improve the product. During Sprint Planning, the Scrum Team selects items from the Product Backlog to work on during the Sprint. These selected items are broken down into tasks and added to the Sprint Backlog, which is a real-time picture of the work the Developers plan to accomplish during the Sprint. So, while user stories themselves are not directly produced in a Sprint, the tasks related to user stories are worked on and completed during the Sprint.

is a user story produced in a sprint ?

Send



Conclusion



Explorer is not a Product but a 'research tool'; a rich Knowledge Base of well-defined Essence Prolog:

- Does not claim to be scalable, robust, reliable, highly available, etc.
 - However, provides very rapid prototyping of ideas, experiments, logical checks, AI integration, and analysis.
 - No user log in, no security. It is a user managed tool.
 - Error checking is minimal but can be easily improved.
 - No Essence Element editing by a Front-end GUI (view) for now but can be added.
 - Minimal GUI, not fancy but easy to extend. Provides a Graphical Framework.
- Most Method CRUD is done by editing code, using a dev tool, e.g., VIM, Visual Studio or Eclipse.
- Nothing is fixed anything can be changed. The tool allows very quick 'method' refactoring as all the Essence material is code. For example, moving elements from one Practice to another is simply cut and paste of that elements Prolog code.
- Manually start Prolog backend to run the Front-End. Simple as: `% swipl prolog/semat_server.pl`
- Manipulating Essence requires Prolog / OO knowledge.
- No current import integration but could be easily added.
- Allows the Essence models to be output as 'English', Java, SQL, or other languages for use in other systems.
- Versions stored by user, e.g., in Git/GitHub. Managed by user, these are all code. Team State is local in DB.
- Clearly separates the View from the Model. Everything is available and traceable back to the raw Prolog facts. With Teamspace it is very hard to trace Views of Entities back to an Origin Model, e.g., Associations may be facts or derived and hard to know, as all you see is the view.
- Allows any new add-hoc queries, logic checks and reports to be created in Prolog.
- Allows exploration of new front ends, UX ideas and GUIs using the APIs.